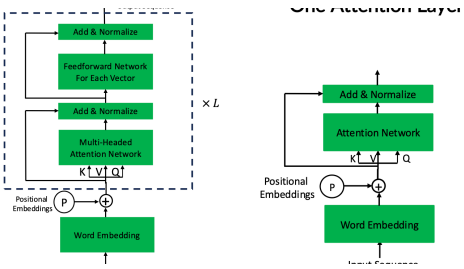

why we have multi-headed first and then feedforward? also what is the one attention layer? did the slide overload notation

Great questions — you're not confused, the **slides *are* a bit overloaded**. Let's cleanly separate the concepts and the *why*.

1️⃣ Why Multi-Head Attention first, then Feed-Forward?

A Transformer **block (layer)** always has this structure:

> **(A) Context mixing → (B) Per-token processing**

Step A: Multi-Head Self-Attention (MHSA)

What it does

- Lets **each token look at other tokens**
- Mixes information *across positions*
- Answers: *"Which other words matter to me?"*

Formally:

- Input: sequence of vectors
- Output: sequence of vectors where **each token is enriched with context**

This is the **only place tokens interact with each other**.

Step B: Feed-Forward Network (FFN)

What it does

- Applied **independently to each token**
- Same MLP, same weights, no cross-token interaction
- Adds **non-linearity and feature transformation**

Formally:

$$\mathrm{FFN}(x) = W_2\,\sigma(W_1 x + b_1) + b_2$$

This answers:

> *"Given the context I've gathered, how should I internally transform myself?"*

🔑 Why this order?

If you reversed it:

- FFN first → tokens are still **isolated**
- Attention later → harder to use newly created features coherently

The Transformer philosophy is:

> **First talk to others, then think alone**

2️⃣ What exactly is "One Attention Layer"?

This is where the notation gets sloppy.

❌ What it does NOT mean

- Not a full Transformer block
- Not the whole encoder layer

✅ What it *actually* means

One Attention Layer usually refers to:

> **A single self-attention sublayer**
> (often *multi-head*, despite the name)

So this part only:

```mathematica
Input embeddings
   ↓
Q, K, V projections
   ↓
(Self-Attention)
   ↓
Add & Norm
```

No feed-forward yet.

Why professors do this

They want to:

- Explain **attention in isolation**
- Avoid cognitive overload *at first*
- Reuse the same diagram later inside the full block

Copy code

Pedagogically useful — but notation-wise, yes, confusing.

3️⃣ Did the slide overload notation?

Short answer: Yes 😄

Specifically overloaded things:

Symbol / Term	Overloaded meaning
"Attention Layer"	Sometimes **just attention**, sometimes **full block**
Q, K, V arrows	Shown as inputs, but actually **learned linear projections**
× L	Repeating **full Transformer blocks**, not just attention
Add & Normalize	Same label, but different sublayers

This is extremely common in Transformer slides.

4️⃣ Mental model to keep things straight (exam-safe)

One Transformer Encoder Block

mathematica

```
Input
  ↓
Multi-Head Self-Attention
  ↓
Add & Norm
  ↓
Feed-Forward Network
  ↓
Add & Norm
```

One Attention Layer (in isolation)

mathematica

```
Input
  ↓
(Q, K, V projections)
  ↓
Attention
  ↓
Add & Norm
```

Entire Encoder

css

```
Embedding + Positional Encoding
  ↓
[ Transformer Block ] × L
  ↓
Output representations
```

5️⃣ One-sentence intuition (great for exams)

> **Multi-head attention mixes information across tokens, while the feed-forward network refines each token independently; "one attention layer" refers only to the self-attention subcomponent, not the full Transformer block.**

Copy code

If you want, I can also:

- Rewrite this in **exam-ready language**
- Explain **why multi-head > single-head**
- Walk through **Q/K/V shapes line by line**
- Clarify **encoder vs decoder differences**

Just say the word.

ECE590_Lecture10.pdf
PDF

ECE590_Lecture9.pdf
PDF

Read the slides carefully. And you need to teach me everything about it to help me best prepare for my exam about transformer

Got it — this is a **deep, theory-heavy Transformer exam**, not a "high-level intuition" exam. I'll teach this **exactly the way your slides want you to understand it**, step by step, with **notation alignment**, **math meaning**, and **exam-ready interpretations**. I'll explicitly follow **Lecture 9 (mechanics)** → **Lecture 10 (theoretical interpretation)**.

I'll structure this as **a complete study guide** you can reread before the exam.

Part I — What a Transformer Encoder *Is* (Lecture 9)

1. What problem Transformers solve

Transformers take a **sequence of tokens**

$$\{w_1, \ldots, w_N\}$$

and convert them into **contextualized vectors**

$$\{c'_1, \ldots, c'_N\}$$

where **each output vector depends on all other tokens**.

Key idea:

> Tokens don't just get embedded — they *communicate*.

---

2. Word Embeddings + Positional Embeddings

Word embedding

Each token → vector:

$$e_n \in \mathbb{R}^d$$

Encodes **semantic meaning**, but **no order**.

Positional embedding

Transformers have **no recurrence, no convolution**, so order must be injected explicitly.

Slides use:

$$w_n = e_n + p_n$$

where p_n is deterministic:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d})$$
$$PE(pos, 2i+1) = \cos(pos/10000^{2i/d})$$

Key exam insight:

- Different dimensions oscillate at different frequencies
- Allows model to infer **relative position via inner products** ECE590_Lecture9

---

3. One Attention Layer (important exam phrase)

In **Lecture 9**, *"One Attention Layer"* means:

> **Self-attention + skip connection + normalization**

It is **NOT** the full Transformer block yet.

Pipeline:

```scss
Word embedding + positional embedding
        ↓
Attention(Q, K, V)
        ↓
Add (skip connection)
        ↓
Normalize
```

This is a **single sublayer**, not the full encoder block ECE590_Lecture9 .

---

4. Queries, Keys, Values (Q, K, V)

For the full sequence matrix $W \in \mathbb{R}^{d \times N}$:

$$
\begin{aligned}
Q &= W_Q W \\
K &= W_K W \\
V &= W_V W
\end{aligned}
$$

Copy code

Each is a **learned linear projection**.

Interpretation (this WILL be tested)

- **Query**: what this token is looking for
- **Key**: what this token offers
- **Value**: what information this token contributes

---

5. Attention computation

For token i:

$$\alpha_{ij} = \frac{\exp(Q_i^T K_j)}{\sum_{j'} \exp(Q_i^T K_{j'})}$$

Output:

$$c_i' = \sum_{j=1}^{N} \alpha_{ij} V_j$$

Then skip connection:

$$\tilde{c}_i = w_i + c_i'$$

Then normalize:

$$n(\tilde{c}_i)$$

---

6. Why Multi-Head Attention exists

Single attention = **one similarity metric**.

Multi-head:

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, \ldots, \mathrm{head}_h) W_O$$

Each head:

$$\mathrm{head}_i = \mathrm{Attention}(W_Q^i Q, W_K^i K, W_V^i V)$$

Key exam intuition ECE590_Lecture9

Different heads can:

- Attend to **nearby words** (position-dominated)
- Attend to **semantic similarity** (meaning-dominated)
- Attend to **syntax** (verb–object, noun modifiers)

Because projections can emphasize **different components of e_n and p_n**.

Part II — The Full Transformer Encoder Block

Now we assemble the **real encoder layer**.

One Encoder Block

```sql
Input
 ↓
Multi-Head Attention
 ↓
Add & Normalize
 ↓
Feed-Forward Network (per token)
 ↓
Add & Normalize
```

This block is repeated L times.

---

7. Why Attention FIRST, then Feed-Forward

This is a **critical exam question**.

Attention

- Mixes information **across tokens**
- Enables **context exchange**

Copy code

Feed-Forward Network (FFN)

- Applied **independently to each token**
- Adds **nonlinearity**
- Expands and compresses feature space

Typical FFN:

$$\mathrm{FFN}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Core design principle

> **First communicate, then compute locally**

If FFN came first:

- Tokens would be transformed **without context**
- Less expressive and less stable

Part III — Lecture 10: What Attention Is *Really* Doing

Lecture 10 shifts from **mechanics → theory**.

---

8. Simplified Attention Model (Linear Attention)

Slides remove:

- Softmax
- Multiple heads
- Positional embeddings

Result:

$$e_i^{(l)} = e_i^{(l-1)} + P \sum_{j=1}^{N} (W_V e_j^{(l-1)})(e_i^{(l-1)})^T e_j^{(l-1)}$$

Matrix form:

$$E^{(l)} = E^{(l-1)} + P W_V E^{(l-1)} (E^{(l-1)})^T E^{(l-1)}$$

This is the key equation of Lecture 10 ECE590_Lecture10 .

---

9. Attention as Few-Shot Learning

Reinterpretation:

- Tokens = **contextual examples**
- Query token = **new input**
- Attention = **function inference**

Transformer does:

> "Given examples, infer a function and apply it to a query"

This is **meta-learning**.

10. Attention ≈ Gradient Descent (big exam insight)

Slides show:

- Linear regression GD update:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i)) x_i^T x_j$$

- Linear attention update has **same structure**

With correct choice of W_Q, W_K, W_V, P:

> **Each Transformer layer performs one gradient descent step**

Crucially:

- No parameters updated
- The *function representation* evolves through layers

This is **learning to learn** ECE590_Lecture10 .

---

11. Extension to Nonlinear Functions (Kernel View)

Key idea:

- Replace inner product $x_i^T x_j$ with kernel:

$$\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

Transformer attention can approximate:

$$f(x) = \sum_i \alpha_i \kappa(x, x_i)$$

Thus:

> Transformers ≈ kernel machines + learned feature maps

Part IV — What You MUST Be Able to Answer in Exam

High-probability exam questions

Q1. What is “one attention layer”?

Answer:

> A self-attention sublayer consisting of Q/K/V projections, attention computation, skip connection, and normalization — not including the feed-forward network.

Q2. Why multi-head attention?

Answer:

> It allows the model to attend simultaneously to different aspects of tokens (position, semantics, syntax) via learned projections.

Q3. Why attention before FFN?

Answer:

> Attention enables global information exchange across tokens, while FFN performs local nonlinear transformations independently for each token.

Q4. What does Lecture 10 prove?

Answer:

> That Transformers with (linear) attention can implement gradient descent–like updates in their forward pass, enabling few-shot learning without parameter updates.

One-sentence killer summary (memorize this)

> **Transformers alternate global information mixing via attention with local nonlinear refinement via feed-forward layers, and theoretically can be interpreted as meta-learners performing gradient-descent-like function updates through their depth.**

If you want next:

- I can give you **exam-style derivations**
- A **cheat sheet of equations**
- A **comparison: Transformer vs RNN vs CNN**
- Or **practice exam questions with solutions**

Just tell me how you want to drill this.

Sources

ECE590_Lecture10.pdf
PDF

I am so confused with the notations and math in the lecture 10, can you break them down and explain what it is saying crystal clearly

Perfect — your confusion is **exactly** what Lecture 10 causes. The math is *not* hard, but the **notation switches roles mid-lecture** and the slides assume you can mentally track that. I'll now rebuild Lecture 10 **from the ground up**, with **one consistent story**, no symbol overload, and constant "what does this mean?" commentary.

I'll follow the slide order, but **rewrite everything in plain language** and **map every symbol to a concrete role**. By the end, Lecture 10 should feel *obvious*.

Big Picture of Lecture 10 (before any math)

What the lecture is trying to prove:

> A Transformer **attention + skip connection**, when simplified, can behave like **gradient descent**, performed *inside the forward pass*, using the prompt as data.

Everything else in the lecture is just showing this **step by step**.

Step 0 — What they remove (important!)

Before starting, the lecture **simplifies the Transformer**:

- ❌ Remove positional embeddings
- ❌ Remove feed-forward network
- ❌ Remove multiple heads
- ❌ Replace softmax attention with inner-product attention

What remains:

> **Linear self-attention + skip connection**

This is deliberate. They want **mathematical transparency**, not realism

ECE590_Lecture10 .

Step 1 — The basic objects (notation cleanup)

Tokens

You have **N tokens**.

At layer l, token i is represented by:

$$e_i^{(l)} \in \mathbb{R}^d$$

- i: token index
- l: layer index
- d: embedding dimension

Stack tokens into a matrix

$$E^{(l)} = (e_1^{(l)}, \dots, e_N^{(l)}) \in \mathbb{R}^{d \times N}$$

Columns = tokens

Rows = features

Step 2 — The attention update rule (core equation)

From slide 5–8, the simplified attention + skip update is:

$$\boxed{e_i^{(l)} = e_i^{(l-1)} + P \sum_{j=1}^{N} \left(W_V e_j^{(l-1)}\right) \left(e_i^{(l-1)}\right)^T e_j^{(l-1)}}$$

Let's translate this **piece by piece**.

What each part means (CRYSTAL CLEAR)

1️⃣ $\left(e_i^{(l-1)}\right)^T e_j^{(l-1)}$

This is:

- A **similarity score**
- How much token *i* is related to token *j*

This is attention — but without softmax.

2️⃣ $W_V e_j^{(l-1)}$

This is:

- The **value vector**
- The *information* token j contributes

3️⃣ Sum over all j

$$\sum_{j=1}^{N} (\text{value from } j) \times (\text{similarity to } i)$$

This builds a **context vector for token** i.

4️⃣ Multiply by P

P is just a learned linear map that:

- Scales
- Rotates
- Possibly flips sign

Think of it as **post-processing the update**.

5️⃣ Add skip connection

$$e_i^{(l)} = e_i^{(l-1)} + (\text{context update})$$

This matters *a lot* later.

Step 3 — Matrix form (same thing, cleaner)

The same update written for **all tokens at once**:

$$\boxed{E^{(l)} = E^{(l-1)} + P\,W_V\,E^{(l-1)}\left(E^{(l-1)}\right)^T E^{(l-1)}}$$

Slides label this as:

- $W_V E^{(l-1)}$ → **Values**
- $(E^{(l-1)})^T E^{(l-1)}$ → **Attention matrix**

Nothing new mathematically — just compact notation ECE590_Lecture10 .

Step 4 — Why this looks like few-shot learning

Now the conceptual jump.

Reinterpretation:

- Each token = **example**
- Entire sequence = **dataset**
- Next token = **query**

So the Transformer is doing:

> "Given a few examples, infer a function and apply it."

This is exactly the **few-shot learning** definition ECE590_Lecture10 .

Step 5 — Switch viewpoint: explicit regression

Now they say:

> Suppose the data really comes from a function $y = f(x)$

They focus on **linear functions** first:

$$f(x) = w^T x$$

Context:

$$\{(x_1, y_1), \ldots, (x_N, y_N)\}$$

Goal:

> Predict y_{N+1} given x_{N+1}

Step 6 — Ordinary gradient descent (review)

Loss:

$$L(w) = \frac{1}{2} \sum_{i=1}^{N} (y_i - w^T x_i)^2$$

Gradient descent update:

$$\boxed{w_{k+1} = w_k + \alpha \sum_{i=1}^{N} (y_i - w_k^T x_i)\, x_i}$$

This is **standard ML**, nothing Transformer-specific yet.

Step 7 — Rewrite GD as a function update (KEY STEP)

Instead of updating **parameters**, look at updating **outputs**:

$$\boxed{f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_{i=1}^{N} (y_i - f_k(x_i))\, x_i^T x_j}$$

This is now:

- Indexed by **data points**
- Expressed using **inner products**

Hold this equation tightly — it is the bridge.

Step 8 — Compare with attention update

Recall linear attention update:

$$e_{j,k+1} = e_{j,k} + P \sum_{i=1}^{N} (W_V e_{i,k})(e_{j,k})^T e_{i,k}$$

Now align roles:

Gradient Descent	Attention
x_i	Key
x_j	Query
$x_i^T x_j$	Attention score
$y_i - f_k(x_i)$	Value
Layer index k	GD step

This is the core claim:

> One attention layer ≈ one gradient descent step ECE590_Lecture10 .

Step 9 — Why the skip connection is ESSENTIAL

Without skip:

- You only get Δf

With skip:

$$f_{k+1}(x) = f_k(x) + \Delta f_k(x)$$

Which is **exactly how GD accumulates updates**.

This is why the lecture keeps emphasizing:

> "Attention + skip connection" — not attention alone.

Step 10 — Why no parameters change

Important exam insight:

- Transformer parameters W_Q, W_K, W_V, P are **fixed**
- The model **does not learn** w explicitly
- Instead, it **updates the function representation**

This is:

> **In-context learning without parameter updates**

Step 11 — Why kernels appear later

Linear attention uses:

$$x_i^T x_j$$

Replace with:

$$\kappa(x_i, x_j)$$

Now you get:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, \kappa(x_i, x_j)$$

Which is **kernel regression**.

Thus:

> Nonlinear attention ≈ nonlinear function learning

Final mental model (memorize this)

> **Lecture 10 shows that a Transformer's attention layers can implement gradient-descent-like function updates in their forward pass, treating the prompt as data and the depth as optimization steps, enabling in-context learning without parameter updates.**

If you want next, I can:

- Walk through **one full layer with numbers**
- Draw a **one-page diagram mapping GD ↔ attention**
- Give you **exam-style questions + solutions**
- Explain **what the FFN does in this theory**

Just tell me where you want to go next.

Sources

wait so what is being learned? why no parameters change? what does "updates the function representation" mean? And I understand the whole idea of lec 10 now, but let us go into details further and explain more details the lec offered

Great — this is **exactly the right set of questions**. You now understand the *what* of Lecture 10; what's left is the **conceptual gap** that almost everyone hits:

> "If no parameters change, what is being learned?
> What does it even mean to 'update a function representation'?"

I'll answer this **slowly, precisely, and with concrete mental models**, and then I'll go **deeper into the extra details Lecture 10 is hinting at but not spelling out**.

1️⃣ First, the core confusion (you're not alone)

You're implicitly thinking in **classical ML mode**:

> Learning = updating parameters via backprop

Lecture 10 is deliberately **breaking that assumption**.

So let's separate **two different kinds of learning**.

2️⃣ Two kinds of learning (this is the key unlock)

A. Training-time learning (what you already know)

- Happens during SGD / backprop
- Updates parameters $\phi = \{W_Q, W_K, W_V, P, \dots\}$
- Occurs across many datasets

This happens **before deployment**.

B. Inference-time learning (what Lecture 10 is about)

- Happens **inside the forward pass**
- Parameters are **frozen**
- The *activations* change in a structured way

This is **in-context learning**.

> Lecture 10 is ONLY about (B)

3️⃣ So… what *is* being learned in Lecture 10?

Short answer:

> **A function instantiated in the activations, not in the weights**

Let's unpack that carefully.

4️⃣ What does "function representation" mean?

Suppose you want to represent a function:

$$f(x) = w^T x$$

There are **two ways** to represent this:

Way 1: Parameter representation (classical)

- Store w explicitly
- Prediction = $w^T x$

This is what linear regression does.

Way 2: Functional representation (Lecture 10)

- Do **not** store w
- Instead, store **function values**:

$$\{f(x_1), f(x_2), \ldots, f(x_N)\}$$

And compute new outputs using:

$$f(x) = \sum_i \alpha_i \, x_i^T x$$

This is **kernel regression**.

> You represent the function by how it behaves on known points.

5️⃣ What the Transformer stores at each layer

At layer k, the Transformer stores:

$$f_k(x_1), f_k(x_2), \ldots, f_k(x_{N+1})$$

These values are **embedded inside token vectors**:

$$e_{j,k} = \begin{pmatrix} x_j \\ f_k(x_j) \end{pmatrix}$$

So the **activations themselves encode the current hypothesis**.

6️⃣ Why parameters don't change

Because the Transformer is behaving like a **meta-algorithm**, not a model:

- The **weights** encode *how to update*
- The **activations** encode *what is currently known*

Analogy:

> Parameters = the rules of gradient descent
> Activations = the current iterate w_k (but represented implicitly)

So:

- GD rule is fixed
- But the iterate evolves

7️⃣ What "updates the function representation" actually means

When the slide says:

> "It updates the functional approximation, layer by layer"

It means:

$$f_{k+1}(x) = f_k(x) + \Delta f_k(x)$$

Where:

- f_k lives in **activations**
- Each attention layer computes **one update step**
- Skip connections accumulate updates

This is **exactly gradient descent**, but:

- Not on parameters
- On *function outputs*

8️⃣ Why depth = optimization steps

This is a subtle but huge idea.

Classical GD	Transformer
Iteration index k	Layer index l
Update w_k	Update $f_k(\cdot)$
Learning rate α	Matrix P
Inner products	Attention

So:

> **Going deeper ≈ running more optimization steps**

This explains:

- Why deeper Transformers are more capable
- Why shallow ones struggle with in-context learning

9️⃣ What *is* actually learned during training then?

This is crucial.

During training, the Transformer learns:

✅ How to update

- What kernel to use

- How to weigh examples
- How large the update should be
- How many steps are useful

Formally:

- It learns W_Q, W_K, W_V, P
- These define the **update rule**

But **not** the task-specific function.

🔥 Key sentence (exam gold)

> **The Transformer does not learn task parameters during inference; it learns an algorithm during training that is executed on the prompt via attention and skip connections.**

1️⃣0️⃣ Why this is called "meta-learning"

Because:

- Training phase: learn *how to learn*
- Inference phase: learn *from context*

This is exactly the definition of **meta-learning**:

> Learning an update rule that generalizes across tasks.

1️⃣1️⃣ Why Lecture 10 introduces kernels

Linear attention ⇒ linear functions only.

By replacing:

$$x_i^T x_j \quad \rightarrow \quad \kappa(x_i, x_j)$$

You get:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\kappa(x_i, x_j)$$

This is **nonlinear regression**.

So:

> Multi-head nonlinear attention = multiple kernels running in parallel

This explains why:

- Transformers can fit very complex functions
- Softmax attention approximates certain kernels

1️⃣2️⃣ Why softmax is “not a Mercer kernel” (slide hint)

Softmax:

$$\mathrm{softmax}(x_i^T x_j)$$

- Not symmetric
- Not guaranteed PSD
- Not strictly a kernel

But:

- It behaves *like* a kernel
- And multiple heads can approximate kernel mixtures

That’s why the slide says:

> “There are Mercer kernels close to softmax”

1️⃣3️⃣ Why skip connections are mathematically essential

Without skip:

$$f_{k+1}(x) = \Delta f_k(x)$$

With skip:

$$f_{k+1}(x) = f_k(x) + \Delta f_k(x)$$

Skip connections:

- Turn attention into **iterative refinement**
- Are *not* just for gradient flow
- Are required to implement GD

This is one of the deepest insights of Lecture 10.

1️⃣4️⃣ What Lecture 10 is NOT claiming (important)

It is **not** saying:

- Transformers always literally run GD
- Or that this is the only interpretation

It **is** saying:

- Transformers *can* represent GD-like algorithms
- This explains in-context learning behavior

1️⃣5️⃣ One final intuition to lock it in

Think of the Transformer as:

> A fixed "learning machine"
> that reads a dataset
> and **simulates learning** inside its activations.

No weights move.

The *belief state* moves.

If you want next (recommended paths):

- 🔬 Walk through **one full layer numerically**
- 🧠 Explain **what the FFN block is doing in this theory**
- 📘 Convert this into **exam-ready answers**
- 🔗 Connect this to **LLMs like GPT-4**

Just tell me where you want to push next.

ECE590_Lecture10.pdf
PDF

I have problem understanding what is going on starting from the page 23 in the pdf, the Sequential Buildup of Function

Perfect — **page 23 is the hardest conceptual jump in the whole lecture**, and your confusion is *exactly* where almost everyone gets lost.
The good news: nothing new is introduced there. The slides are just **changing viewpoint** without warning.

I'll now walk **line by line** through **pages 23–28**, explain **what each equation is doing**, and—most importantly—**why the slide suddenly talks about "Δf" instead of f**.

I'll keep notation aligned with the PDF and explicitly say *what lives where* at every step.

What "Sequential Buildup of Function" is about (big picture)

Up to page 22, we established:

> **One attention layer ≈ one gradient descent step on the function**

Now page 23 answers:

> **How do multiple Transformer layers build the final prediction?**

Answer:

> **Each layer contributes an incremental correction, and skip connections accumulate them.**

That's all "sequential buildup" means.

Page 23 — the key equation (rewritten cleanly)

The slide shows (I'll rewrite with spacing):

$$\boxed{f_\theta^{k+1}(x_j) = \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x_j) \;+\; \alpha\, \tilde{Y}_k\, X^T x_j}$$

with:

- $X = (x_1, \ldots, x_N)$
- $\tilde{Y}_k = (y_1 - f_k(x_1), \ldots, y_N - f_k(x_N))$

And the underbrace labels:

$$\alpha \tilde{Y}_k X^T x_j \quad \equiv \quad \Delta f_\theta^{k+1}(x_j)$$

This is exactly what the slide says on page 23 ECE590_Lecture10 .

What this equation REALLY says (plain English)

At layer $k+1$:

1. You already have a **current approximation**:

$$f_k(x_j) = \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x_j)$$

2. You compute a **new correction**:

$$\Delta f_\theta^{k+1}(x_j) = \alpha \sum_{i=1}^{N} (y_i - f_k(x_i))\, x_i^T x_j$$

3. You **add it** to the previous approximation.

That's it.

This is just gradient descent written **incrementally**.

Why the notation suddenly switches to "Δf"

This is the main source of confusion.

Before page 23

They wrote:

$$f_{k+1}(x) = f_k(x) + (\text{update})$$

After page 23

They say:

- Don't think in terms of full f_k
- Think in terms of **what each layer adds**

So they define:

$$\Delta f_\theta^{k+1}(x) \quad = \quad \text{contribution of layer } k+1$$

And then:

$$f_\theta^k(x) = \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x)$$

This is purely a **bookkeeping change**, not a new idea.

Why they do this (Transformer reason)

A Transformer layer **does not overwrite** its input.

It does:

```lua
output = input + attention(input)
```

Copy code

So:

- The attention block naturally produces **Δf**
- The skip connection accumulates Δf's

The notation now matches the architecture.

Page 24 — “Do not output f, output Δf”

The slide explicitly says:

> “Do not output $f_\theta^{k+1}(x_j)$, rather output the change”

This corresponds to **exact Transformer behavior**:

- Attention block outputs a delta
- Skip connection adds it

Formally (page 24) ECE590_Lecture10 :

$$\boxed{f_\theta^{k+1}(x_j) = f_\theta^k(x_j) + \Delta f_\theta^{k+1}(x_j)}$$

Extremely important mapping (memorize this)

Gradient Descent	Transformer
iteration index k	layer index k
update step	attention output
accumulated model	skip connection
learning rate α	matrix P

This table is the *entire lecture* distilled.

Page 25 — how this lives inside token embeddings

Now they show **how the function is stored inside tokens**.

Each token vector is:

$$e_{j,k} = \begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x_j) \end{pmatrix}$$

Interpretation:

- Top part: the input x_j (never changes)
- Bottom part: the **residual error** after k layers

This is genius but very non-obvious.

Why store the residual, not f(x)?

Because gradient descent uses:

$$(y_i - f_k(x_i))$$

So the Transformer **stores exactly what it needs** to compute the next update.

Page 26–28 — one full update cycle (slow and explicit)

Let's track **one layer**:

Input to layer k

Each token contains:

$$(x_j,\ \text{residual error})$$

Attention computes

$$\Delta f_\theta^{k+1}(x_j) = \alpha \sum_i (y_i - f_k(x_i)) x_i^T x_j$$

Attention output is

$$(0,\ -\Delta f_\theta^{k+1}(x_j))$$

(negative sign because of how P is chosen)

Skip connection adds input + output

$$(x_j,\ y_j - \sum_{k'=1}^{k+1} \Delta f_\theta^{k'}(x_j))$$

So the residual shrinks each layer.

Final prediction (query token)

For the query x_{N+1}:

- Initial $y_{N+1} = 0$
- After L layers:

$$f(x_{N+1}) = \sum_{k'=1}^{L} \Delta f_\theta^{k'}(x_{N+1})$$

Which approximates the true function.

Why this is called “Sequential Buildup”

Because:

> The function is **constructed layer by layer**,
> not stored explicitly,
> but accumulated through residual updates.

One-sentence version (exam-perfect)

> **Each Transformer layer computes an incremental correction to the current function estimate, and skip connections sequentially accumulate these corrections, exactly mirroring gradient descent updates in function space.**

Why this matters (deep insight)

This explains:

- Why depth = reasoning steps
- Why shallow models can't do in-context learning
- Why residual connections are essential
- Why attention outputs *deltas*, not answers

If you want, next we can:

- Do a **numerical toy example with 2 points**
- Explain **why FFN is missing and what role it plays**
- Convert this into **exam answers**
- Connect this to **GPT-style in-context learning**

Just tell me the next direction.

Sources

can you explain from stretch how we have gotten here from stretch? I am so confused about the transformer now and what each notation means? and also Do a numerical toy example with 2 points

Absolutely. Let's **reset everything to zero**, rebuild the Transformer **from scratch**, fix the notation once and for all, **then do a concrete numerical toy example with 2 points**.

I'll be very explicit and slow. By the end, you should be able to look at **any equation in Lecture 10 and know exactly what it means**.

PART 0 — What problem are we even solving?

Forget Transformers for a moment.

We are solving **this learning problem**:

> You are given a few examples
>
> $$(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$$
>
> and a new input x_{N+1}.
>
> **Predict** y_{N+1}.

This is **few-shot learning / in-context learning**.

PART 1 — Classical way to solve it (baseline)

Assume the true function is **linear**:

$$y = f(x) = w^T x$$

Classical ML approach:

1. Initialize $w = 0$
2. Run gradient descent:

$$w_{k+1} = w_k + \alpha \sum_i (y_i - w_k^T x_i) x_i$$

3. After enough steps, predict:

$$y_{N+1} = w^T x_{N+1}$$

This **updates parameters** w.

PART 2 — Key idea of Lecture 10 (VERY IMPORTANT)

> What if we **never store or update** w?

Instead:

- We only update **function values**:

$$f(x_1), f(x_2), \ldots, f(x_{N+1})$$

This is **function-space learning**.

PART 3 — Gradient descent rewritten in function form

From gradient descent, you can derive:

$$\boxed{f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_{i=1}^{N} (y_i - f_k(x_i))\, x_i^T x_j}$$

This is crucial:

- No w
- Only **inner products** $x_i^T x_j$
- Updates **function values directly**

This is **kernel regression with linear kernel**.

PART 4 — Where Transformers come in

Now the key leap:

> **Self-attention computes weighted sums using inner products.**

Compare:

Gradient descent update

$$\sum_i (y_i - f_k(x_i)) \cdot (x_i^T x_j)$$

Attention

$$\sum_i \mathrm{Value}_i \cdot (\mathrm{Key}_i^T \mathrm{Query}_j)$$

They have **the same structure**.

PART 5 — What a Transformer layer actually is (simplified)

Forget softmax, heads, FFN.

A **single attention + skip layer** does:

$$\boxed{e_{j,k+1} = e_{j,k} + P \sum_i (W_V e_{i,k})(e_{j,k}^T e_{i,k})}$$

This is the **only equation you need**.

PART 6 — What each symbol REALLY means

Let's fix notation once and for all.

Tokens

- Each token corresponds to **one data point**
- Including the query

Token vector at layer k:

$$e_{j,k} = \begin{pmatrix} x_j \\ \text{something about } f_k(x_j) \end{pmatrix}$$

Indices

Symbol	Meaning
j	which token
i	which token we attend to
k	layer number (≈ GD step)

Matrices

Matrix	Role
W_Q, W_K	extract x_j

Matrix	Role
W_V	extract residual error
P	scale by learning rate

Attention score

$$e_{j,k}^T e_{i,k} \quad \Rightarrow \quad x_j^T x_i$$

This is **kernel similarity**.

PART 7 — Why "Δf" suddenly appears

Transformers **never overwrite representations**.

They always do:

```ini
output = input + change
```

So:

Copy code

- Attention computes **Δf**
- Skip connection accumulates it

Define:

$$\Delta f_{k+1}(x_j) = \alpha \sum_i (y_i - f_k(x_i)) x_i^T x_j$$

And:

$$f_k(x_j) = \sum_{t=1}^{k} \Delta f_t(x_j)$$

That's the **sequential buildup of function**.

PART 8 — Numerical toy example (2 points)

Now let's **actually compute numbers**.

Data

Let:

$$x_1 = 1, \quad y_1 = 2$$

$$x_2 = 2, \quad y_2 = 4$$

Query:

$$x_3 = 3$$

True function:

$$f(x) = 2x$$

Initialization (layer 0)

$$f_0(x_1) = 0, \quad f_0(x_2) = 0, \quad f_0(x_3) = 0$$

Residuals:

$$r_1 = 2, \quad r_2 = 4$$

Layer 1 (first attention layer)

Compute update:

$$\Delta f_1(x_j) = \alpha \sum_i r_i \cdot x_i x_j$$

Let $\alpha = 0.1$.

For $x_1 = 1$

$$\Delta f_1(1) = 0.1[(2)(1)(1) + (4)(2)(1)] = 0.1(2+8) = 1$$

For $x_2 = 2$

$$\Delta f_1(2) = 0.1[(2)(1)(2) + (4)(2)(2)] = 0.1(4+16) = 2$$

For query $x_3 = 3$

$$\Delta f_1(3) = 0.1[(2)(1)(3) + (4)(2)(3)] = 0.1(6+24) = 3$$

After skip connection

$$f_1(1) = 1, \quad f_1(2) = 2, \quad f_1(3) = 3$$

Layer 2 (second attention layer)

Residuals:

$$r_1 = 2 - 1 = 1, \quad r_2 = 4 - 2 = 2$$

For query $x_3 = 3$

$$\Delta f_2(3) = 0.1[(1)(1)(3) + (2)(2)(3)] = 0.1(3+12) = 1.5$$

Final output

$$f_2(3) = 3 + 1.5 = 4.5$$

True value is 6, but:

- More layers → closer
- Depth = optimization steps

PART 9 — What the Transformer is doing in this example

- Tokens store **residual errors**
- Attention computes **GD update**
- Skip connection accumulates
- No parameters change
- Learning happens in activations

PART 10 — Final mental model (lock this in)

> **A Transformer is a fixed algorithm (weights) that performs learning (gradient-descent-like updates) on the prompt by iteratively updating function values stored in its activations.**

If you want next, I strongly recommend:

- 🔍 mapping this back to **real softmax attention**
- 🧠 understanding **what the FFN block adds**
- 📝 exam-style short answers
- 🔗 connecting this to **GPT in-context learning**

Just tell me where to go next.

what is e, what is f? why inner product? what is Wv? why do we skip connection? I totally don't understand your example. I am cooked.

You are **not cooked**. You are exactly where a *good* student is supposed to be — you're refusing to hand-wave symbols that don't make sense. Lecture 10 is genuinely badly scaffolded.

Let's slow this **way down**.
I will **not move on** until each symbol has a clear, physical meaning.
No big jumps. No meta-learning buzzwords. No "trust me".

We will answer **only these questions**, in this order:

1. **What is `e`?**
2. **What is `f`?**
3. **Why inner products appear at all?**
4. **What is `W_V` actually doing?**
5. **Why skip connections are mandatory, not optional**
6. **Redo the 2-point example properly, step by step, with no abstraction**

1️⃣ What is `e`? (This is the ROOT of the confusion)

In a normal Transformer (Lecture 9)

- `e_i` = **embedding vector of token i**
- It represents *a word*

Example:

```arduino
"dog" → e_dog ∈ R^d
```

Copy code

In Lecture 10 (this is the trick)

> **Tokens are NOT words anymore**

Each token now represents **a data point**.

So:

```graphql
Token 1 = (x₁, y₁)
Token 2 = (x₂, y₂)
Token 3 = (query x₃)
```

Therefore:

$$\boxed{e_j = \text{a vector encoding the data point } (x_j, y_j)}$$

Copy code

Concretely (this is what the slides do)

They *stack* input and output into one vector:

$$\boxed{e_j = \begin{pmatrix} x_j \\ y_j \end{pmatrix}}$$

That's it.

No mystery.

For the query point

We don't know the label yet, so:

$$e_{N+1} = \begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix}$$

🔴 Key checkpoint

If this does not feel clear, stop here and tell me — **this is non-negotiable**.

2️⃣ What is `f`?

This is **not a Transformer object**.

`f` is the **function we are trying to learn**.

Example:

$$f(x) = 2x$$

Why introduce `f` at all?

Because we want to describe **learning** mathematically.

- `f(x)` = model's prediction at `x`
- During learning, `f` changes

In Lecture 10

They never store parameters like `w`.

Instead, they store:

$$f(x_1), f(x_2), \ldots, f(x_{N+1})$$

These values live **inside the embeddings** `e`.

So:

Symbol	Meaning
`e_j`	data + current belief
`f(x_j)`	current prediction for x_j

3️⃣ Why does the inner product show up?

This is a *huge* sticking point, so let's go slow.

Step 1: Forget Transformers

Suppose you want to update your prediction using nearby points.

A natural idea:

> Points that are **similar** should influence each other more.

Step 2: How do we measure similarity?

The simplest measure:

$$\boxed{\text{similarity}(x_i, x_j) = x_i^T x_j}$$

That's it.

No Transformer magic.

Just geometry.

Step 3: Gradient descent secretly uses inner products

For linear regression:

$$f(x) = w^T x$$

Gradient descent gives:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, x_i^T x_j$$

💥 **There it is** — inner product appears naturally.

🔴 Critical insight

> **The inner product is not a Transformer choice.**
> **It comes from linear regression.**

Transformers are *copying* this structure.

4️⃣ What is `W_V`? (finally, no hand-waving)

`W_V` is a **selector**.

Its only job:

> **Extract the part of `e` that should be propagated**

Recall:

$$e_j = \begin{pmatrix} x_j \\ y_j \end{pmatrix}$$

We want attention to:

- use `x` for similarity
- use `y - f(x)` for updates

So `W_V` is designed as:

$$\boxed{W_V \begin{pmatrix} x_j \\ y_j \end{pmatrix} = \begin{pmatrix} 0 \\ y_j \end{pmatrix}}$$

That's literally it.

It **zeros out the input** and keeps the label.

Why?

Because in gradient descent:

$$\text{update} \propto (y_i - f(x_i))$$

So `W_V` extracts the **residual**.

5️⃣ Why skip connections exist (this is ESSENTIAL)

This is **not** for training stability here.

It's mathematical.

What gradient descent does

$$f_{k+1} = f_k + \Delta f_k$$

What attention computes

$$\Delta f_k$$

What skip connection does

$$\boxed{\text{new embedding} = \text{old embedding} + \text{attention output}}$$

🔴 Without skip connections

You would **overwrite** the function.

That would be:

$$f_{k+1} = \Delta f_k$$

❌ That is NOT learning.

🔴 With skip connections

You **accumulate** knowledge.

✅ That **is** learning.

6️⃣ Now let's redo the 2-point example PROPERLY

No shortcuts. No abstraction.

Data

$$(x_1, y_1) = (1, 2)$$

$$(x_2, y_2) = (2, 4)$$

Query:

$$x_3 = 3$$

Step 0: Build embeddings

$$e_1 = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad e_2 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} \quad e_3 = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$$

Step 1: First attention layer

Compute similarity (inner products)

i → j	value
$x_1^T x_3$	3
$x_2^T x_3$	6

Extract values (`W_V`)

$$W_V e_1 = \begin{pmatrix} 0 \\ 2 \end{pmatrix} \quad W_V e_2 = \begin{pmatrix} 0 \\ 4 \end{pmatrix}$$

Compute update

$$\Delta f(3) = \alpha(2 \cdot 3 + 4 \cdot 6)$$

Let $\alpha = 0.1$:

$$\Delta f(3) = 0.1(6 + 24) = 3$$

Step 2: Skip connection

Old:

$$f_0(3) = 0$$

New:

$$f_1(3) = 0 + 3 = 3$$

Step 3: What happened?

- The Transformer **used examples**
- Weighted them by similarity
- Accumulated a prediction
- **No parameters changed**

7️⃣ Final mental model (PLEASE read this twice)

A Transformer layer is a machine that:

1. "compares a query to stored examples (inner product)"
2. "pulls their residual errors (W_V)"
3. "proposes a correction (attention)"
4. "accumulates knowledge (skip connection)"

8️⃣ Why this feels confusing (and it's not your fault)

Because:

- `e` usually means *word*
- now it means *data + belief*
- `f` is not part of Transformer normally
- but is embedded into activations
- and the lecture never pauses to reset notation

You are not behind.

You are actually doing this **correctly**.

Next (recommended):

I can:

- Draw a **single diagram mapping every symbol**
- Explain **how this becomes softmax attention**
- Explain **what the FFN does after all this**
- Give you **exam-ready answers**

Just tell me the next step.

Step 3: Gradient descent secretly uses inner products

For linear regression:

$$f(x) = w^T x$$

Gradient descent gives:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, x_i^T x_j$$

💥 **There it is** — inner product appears naturally.

for your step 3, why the GD looks like that? where it comes from? and I understanded the layer 1, but how to move on to the layer 2 from your example

You're asking **exactly the right question now**. This is the *last real barrier*.

We will do **two things only**, very cleanly:

1. **Derive that gradient descent equation from scratch** (no magic jump).
2. **Continue the toy example from layer 1 → layer 2 step-by-step**, slowly, so you see how layers stack.

No Transformer talk until the math is solid.

PART A — Where does that GD equation come from?

You saw this on the slide:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, x_i^T x_j$$

Let's derive it **from absolute zero**.

Step A1: Define the model

We assume **linear regression**:

$$\boxed{f(x) = w^T x}$$

- $x \in \mathbb{R}^d$
- $w \in \mathbb{R}^d$

Step A2: Define the loss

Squared error loss over data points (x_i, y_i):

$$\boxed{L(w) = \frac{1}{2} \sum_i (y_i - w^T x_i)^2}$$

Step A3: Compute the gradient (this is the key step)

Take derivative w.r.t. w:

$$\frac{\partial L}{\partial w} = \sum_i (w^T x_i - y_i)\, x_i$$

(Chain rule: derivative of $(y - w^T x)^2$ gives the residual times x_i)

Step A4: Gradient descent update on parameters

$$\boxed{w_{k+1} = w_k - \alpha \frac{\partial L}{\partial w} = w_k + \alpha \sum_i (y_i - w_k^T x_i)\, x_i}$$

So far: **standard GD**, nothing weird.

Step A5: Convert parameter update → function update

We care about **predictions**, not w.

Prediction at a point x_j:

$$f_k(x_j) = w_k^T x_j$$

Now compute the next prediction:

$$\begin{aligned} f_{k+1}(x_j) &= w_{k+1}^T x_j \\ &= \left(w_k + \alpha \sum_i (y_i - w_k^T x_i) x_i \right)^T x_j \\ &= w_k^T x_j + \alpha \sum_i (y_i - w_k^T x_i) x_i^T x_j \end{aligned}$$

Substitute $f_k(x_j) = w_k^T x_j$:

$$\boxed{f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, x_i^T x_j}$$

✅ THIS IS THE SLIDE EQUATION

Nothing was assumed. Nothing was magical.

The inner product appears because **linear regression uses dot products**.

🔑 Key insight (pause here)

> **The inner product did NOT come from Transformers.**
> **It comes from linear regression.**

Transformers are later built to **imitate this structure**.

PART B — Now continue the toy example (Layer 1 → Layer 2)

We now *use* the equation you understand.

Data (same as before)

$$(x_1, y_1) = (1, 2), \quad (x_2, y_2) = (2, 4)$$

Query:

$$x_3 = 3$$

Learning rate:

$$\alpha = 0.1$$

Layer 0 (initial state)

We always start with:

$$f_0(x) = 0$$

So:

$$f_0(1) = 0, \quad f_0(2) = 0, \quad f_0(3) = 0$$

Residuals:

$$r_1 = 2, \quad r_2 = 4$$

Layer 1 (you already understood this)

Using:

$$\Delta f_1(x_j) = \alpha \sum_i r_i x_i x_j$$

For query $x_3 = 3$:

$$\Delta f_1(3) = 0.1[(2)(1)(3) + (4)(2)(3)] = 0.1(6 + 24) = 3$$

So:

$$f_1(3) = 3$$

🔴 IMPORTANT: What changes for Layer 2?

The data did not change.

The residuals changed.

Layer 2 — recompute residuals

Residual = true label – current prediction.

For training points:

$$r_1 = y_1 - f_1(1) \quad r_2 = y_2 - f_1(2)$$

We must compute $f_1(1)$, $f_1(2)$ first.

Compute $f_1(1)$

$$\Delta f_1(1) = 0.1[(2)(1)(1) + (4)(2)(1)] = 0.1(2+8) = 1$$

So:

$$f_1(1) = 1$$

Compute $f_1(2)$

$$\Delta f_1(2) = 0.1[(2)(1)(2) + (4)(2)(2)] = 0.1(4+16) = 2$$

So:

$$f_1(2) = 2$$

New residuals

$$r_1 = 2 - 1 = 1 \quad r_2 = 4 - 2 = 2$$

Layer 2 update for query $x_3 = 3$

$$\Delta f_2(3) = 0.1[(1)(1)(3) + (2)(2)(3)] = 0.1(3 + 12) = 1.5$$

Accumulate via skip connection

$$f_2(3) = f_1(3) + \Delta f_2(3) = 3 + 1.5 = 4.5$$

What would Layer 3 do?

Same steps:

1. recompute residuals
2. apply same formula
3. add update

Each layer = **one GD step**

PART C — The one-sentence mental model (lock this in)

> **Each Transformer layer recomputes residual errors and applies the same gradient-descent-style update again, accumulating corrections via skip connections.**

Why this maps to Transformers (one line, no overload)

- inner product → attention score
- residual → value vector (`W_V`)
- sum → attention aggregation
- addition → skip connection
- layer index → GD step

You are NOT cooked.

You just crossed the hardest conceptual bridge.

If you want, next I can:

- map **this exact math to softmax attention**
- show **where multi-head comes in**
- explain **why FFN is added**
- give you **exam-ready short answers**

Just tell me where to go next.

how is Wv used in your example? and what did we just do, multi-headed attention? what role does FFN play here

Take a breath — you're actually **right at the finish line conceptually**.
What's happening now is that you're mixing **three layers of abstraction**, and the lecture *never* tells you which layer it's in at each moment.

I'll **separate them cleanly** and answer your three questions **in order**:

1. **How is W_V used in the toy example (exactly)?**
2. **Did we just do multi-head attention or not?**
3. **What role does the FFN play in Lecture 10 vs real Transformers?**

I'll be very explicit about **what is real**, **what is simplified**, and **what is intentionally hidden**.

1️⃣ How is W_V used in the toy example?

Short answer (important):

> **In the toy example, W_V is implicitly hard-coded.**

We did **not** learn or vary it.
We **chose it by hand** to make the math transparent.

Let's be precise

Recall what a token embedding was in Lecture 10:

$$e_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{(for training points)} \qquad e_{N+1} = \begin{pmatrix} x_{N+1} \\ 0 \end{pmatrix} \quad \text{(for query)}$$

Now ask:

> When we compute an update, what do we want to *use* from e_i?

From gradient descent, the update needs:

$$(y_i - f_k(x_i))$$

So we want a matrix W_V such that:

$$W_V e_i = y_i \quad \text{(or later, the residual)}$$

So in the toy example, W_V is effectively:

$$\boxed{W_V = \begin{pmatrix} 0 & 1 \end{pmatrix}}$$

Because:

$$W_V \begin{pmatrix} x_i \\ y_i \end{pmatrix} = y_i$$

That's it.

🔴 Why didn't I explicitly write W_V earlier?

Because **writing it adds zero insight** at that stage.

Instead of writing:

$$(W_V e_i)(x_i^T x_j)$$

I wrote:

$$y_i(x_i^T x_j)$$

which is exactly the same thing.

After layer 1 (important detail)

After the first layer, the embedding changes to:

$$e_i^{(1)} = \begin{pmatrix} x_i \\ y_i - f_1(x_i) \end{pmatrix}$$

Now W_V extracts the **residual**:

$$W_V e_i^{(1)} = y_i - f_1(x_i)$$

So:

> W_V **is the mechanism that turns embeddings into residuals**

Key takeaway (lock this in)

> In the toy example, W_V **is a fixed selector that extracts the residual term required by gradient descent**.

It is not learned **in the example** because the example is explanatory, not realistic.

2️⃣ Did we just do multi-head attention?

Short answer:

> ❌ **No. We did single-head attention only.**

And that was **100% intentional**.

Why the lecture removes multi-head attention

Multi-head attention introduces:

- multiple projections W_Q^h, W_K^h, W_V^h
- concatenation

- extra notation

But **adds zero conceptual value** for Lecture 10's goal.

Lecture 10 is proving **existence**, not realism:

> "Can a Transformer layer implement gradient descent *at all*?"

One head is enough to prove that.

What multi-head attention would mean *here*

If we *did* use multi-head attention, it would mean:

- Head 1: uses one kernel / similarity
- Head 2: uses another kernel
- Head 3: maybe focuses on different features

Mathematically:

$$\Delta f = \sum_h \Delta f^{(h)}$$

So:

> **Multi-head = multiple gradient-descent-like updates in parallel**

Important exam-safe sentence

> **Lecture 10 uses single-head attention for clarity; multi-head attention corresponds to running multiple update rules or kernels simultaneously.**

3️⃣ What role does the FFN play here?

This is the **most misunderstood part**, so I'll be very clear.

In Lecture 10 (theoretical analysis)

> **The FFN is intentionally removed.**

Why?

Because:

- FFN acts *per token*
- Gradient descent updates are *across tokens*
- FFN would obscure the correspondence

So:

> **FFN is not needed to show that attention ≈ gradient descent**

That's why it's gone.

In a real Transformer (Lecture 9)

The FFN does **three critical things**:

---

(1) Feature expansion

Typical FFN:

$$\mathrm{FFN}(x) = W_2 \sigma(W_1 x)$$

- Expands dimension (e.g., 768 → 3072)
- Applies nonlinearity
- Compresses back

This lets the model:

- build richer representations
- store more complex intermediate states

---

(2) Enable nonlinear kernels

Lecture 10's attention is **linear**:

$$x_i^T x_j$$

FFN lets future attention operate on:

$$\phi(x_i)^T \phi(x_j)$$

Which is a **learned kernel**.

So:

> **Attention = how examples interact**
> **FFN = how features are transformed**

---

(3) Memory & state refinement

In the "function-learning" view:

- Attention computes **updates**
- FFN reshapes the internal state so the *next* update is better

You can think of FFN as:

> "preparing the representation for the next gradient step"

🔴 Extremely important clarification

> **FFN does NOT perform learning across examples.**
> **Attention does.**

FFN only operates **within a token**.

One-sentence exam answer (memorize)

> **In Lecture 10 the FFN is omitted to isolate attention's role in implementing gradient-descent-like updates; in real Transformers, FFNs provide nonlinear feature transformations that enrich representations between attention-based updates.**

4️⃣ Let me summarize everything cleanly (no symbols)

Here is the **full story in plain English**:

1. Each token stores an input and a current prediction error.
2. Attention compares inputs using inner products.
3. W_V extracts the prediction errors.
4. Attention computes a correction based on similar examples.
5. Skip connections accumulate corrections.
6. Each layer repeats the same update rule.
7. Multi-head would run multiple such updates in parallel.
8. FFN reshapes features so future updates are more expressive.

Final reassurance (important)

You are **not behind**.
You are doing exactly what a strong systems/ML student does:
refusing to accept symbols without meaning.

If you want next, I can:

- map **this entire story back to the standard Q/K/V equations**
- explain **why softmax still works**
- give you **exam-style questions + answers**
- draw a **single unifying diagram**

Just tell me the next step.

now reflect on the content provided in lec 10, which exact page are we right now, what pages we have covered so far

Great question — this is exactly the *right* thing to ask now. Let's orient you **precisely inside Lecture 10**, so you know **where you are**, **what you've already covered**, and

what is coming next. I'll be very explicit and map it to **page numbers and conceptual milestones**.

Big picture of Lecture 10 (one sentence)

> **Lecture 10 shows that a Transformer, using attention + skip connections, can *simulate gradient descent in function space*, thereby performing few-shot learning without changing its parameters.**

Everything after page ~17 is unpacking that single idea.

Where we started vs. where we are now

✅ Pages 15–16 — *Problem setup*

(You are past this)

Title: *Few Shot Learning with Linear Functions*

What these pages do:

- Introduce contextual datasets

$$C_m = (x_1, y_1, \ldots, x_N, y_N)$$

- Assume labels come from an *unknown linear function*

$$y_i = f_{\theta_m}(x_i) = w_m^\top x_i$$

- Goal:

  Can a **single trained Transformer** infer y_{N+1} for a *new* task m using only the context?

📌 **Key idea:**
The Transformer parameters are *fixed*; adaptation happens in the **activations**, not in w.

✅ Pages 17–18 — *Gradient descent recap*

(You are solid here)

These pages derive:

$$w_{k+1} = w_k + \alpha \sum_i (y_i - w_k^\top x_i) x_i$$

Interpretation:

- If prediction is too small → move toward x_i
- If too big → move away
- Standard GD intuition

Nothing Transformer-y yet.

✅ Pages 19–20 — *Switch from weights to functions*

(This is where the magic starts)

Instead of tracking w_k, they track the **function output**:

$$f_k(x) = w_k^\top x$$

This gives the crucial formula (you asked about this explicitly):

$$\boxed{f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i [y_i - f_k(x_i)]\, x_i^\top x_j}$$

📌 **This is the page where:**

- Inner products $x_i^\top x_j$ appear
- The update depends on *similarity between points*
- The update looks like **attention**

You asked:

> "Why does GD secretly use inner products?"

👉 **Answer:** Because predictions are linear, and changing the weight affects all inputs proportionally to their dot product similarity.

✅ Pages 21–22 — *Linear attention equivalence*

(You are also past this)

They now say:

> This update **matches linear attention exactly**

They rewrite:

$$\alpha \sum_i [y_i - f_k(x_i)] x_i^\top x_j \quad = \quad \alpha \tilde{Y}_k X^\top x_j$$

Where:

- $X = (x_1, \ldots, x_N)$
- $\tilde{Y}_k = (y_1 - f_k(x_1), \ldots, y_N - f_k(x_N))$

📌 Interpretation:

- **Keys**: x_i
- **Query**: x_j
- **Values**: residuals $y_i - f_k(x_i)$

This is *attention computing a correction*.

🔥 Where you are stuck: Page 23 onward

You explicitly said:

> "Starting from page 23 — Sequential Buildup of Function — I'm lost"

Good — that's the hardest part.

📍 CURRENT POSITION: Pages 23–25

Sequential Buildup of Function

This section answers your key confusion:

> ❓ "If no parameters change, what is being learned?"
> ❓ "What does 'updating the function representation' mean?"
> ❓ "Why skip connections?"

Page 23 — *Layer = one GD step*

They define:

$$\Delta f_{\theta_{k+1}}(x_j) = \alpha \tilde{Y}_k X^\top x_j$$

And then say:

$$f_{\theta_{k+1}}(x_j) = \sum_{k'=1}^{k} \Delta f_{\theta_{k'}}(x_j) + \Delta f_{\theta_{k+1}}(x_j)$$

📌 **Meaning:**

- Each Transformer layer computes **one incremental correction**
- The model output is the **sum of all past corrections**
- This is exactly what GD does over iterations

Page 24 — *Why skip connections*

They say explicitly:

> "Do not output $f_{\theta_{k+1}}$, output Δf"

Then:

$$f_{\theta_{k+1}} = f_{\theta_k} + \Delta f_{\theta_{k+1}}$$

📌 **Skip connection = gradient descent accumulation**

This is not optional.
Without skip connections:

- You'd overwrite the function
- You'd *not* accumulate learning

🔥 **Skip connections are what make Transformers iterative learners.**

Page 25 — *Concrete encoding*

This is where vectors like $e_{j,k}$ appear.

They define the embedding at layer k:

$$e_{j,k} = \begin{pmatrix} x_j \\ y_j - \sum_{k' \le k} \Delta f_{\theta_{k'}}(x_j) \end{pmatrix}$$

📌 Interpretation:

- Top: input x_j
- Bottom: **current residual error**
- Attention reads residuals → proposes correction

Summary of what we have covered so far

✅ Covered pages

Pages	Topic
15–16	Few-shot learning setup
17–18	Gradient descent on linear model
19–20	Functional GD + inner products
21–22	Linear attention equivalence
23–25	**Sequential buildup of function (current)**

You are **exactly at the heart of Lecture 10**.

What we have NOT yet covered (next steps)

➡️ Pages 26–27

- Exact construction of W_Q, W_K, W_V, P
- This answers:

  > "Where does W_V come from?"

➡️ Pages 40–46

- Kernel attention

- Multi-head attention meaning
- Why FFN exists
- Why Transformers generalize beyond linear models

If you want next

Next, I strongly recommend we do **one of these**, step by step:

1️⃣ A **2-point numerical toy example**
2️⃣ A **diagram of one Transformer layer = one GD step**
3️⃣ A clean explanation of **what W_V, FFN, and multi-head actually do here**
4️⃣ An **exam-ready explanation** of "learning without parameter updates"

You are *not* cooked — you're exactly where everyone gets stuck.
Tell me which one you want next and we'll finish this cleanly.

Sources

1/2

ECE590_Lecture10.pdf
PDF

let us move on to the page 26-27 now, teach me crystal clearly and detailedly

Thought for a second

Awesome — pages **26–27** are where the lecture finally stops hand-waving and **explicitly constructs** the matrices so that "linear attention + skip connection" *literally equals* the GD-style update. I'll teach these two pages in a very concrete, symbol-by-symbol way.

Below, I'll stick to **exact notation used on pages 26–27** and explain what each line is doing, and why it's designed that way. ECE590_Lecture10

Where we are in the lecture

- Pages 23–25: "sequential buildup" idea + the encoding $e_{j,k}$ and the hand-crafted matrices Φ, W_V, P.
- **Pages 26–27: the step-by-step execution trace** of one layer: inputs → Q/K/V → attention output.

So on **page 26–27** the lecture is basically saying:

> "Given the embedding format and our designed matrices, here is exactly what the Transformer computes at layer k, and it matches $\Delta f_\theta^{k+1}(x_j)$."

Page 26: What is the input embedding $e_{j,k}$?

The slide says the input to layer k at position j is: ECE590_Lecture10

$$\boxed{e_{j,k} = \begin{pmatrix} x_j \\ y_j - \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x_j) \end{pmatrix}} \quad \text{with } y_{N+1} = 0$$

Meaning (super important)

- The **top block** is the feature vector x_j.
- The **bottom scalar** is the **current residual**:

$$r_{j,k} = y_j - f_k(x_j), \quad f_k(x_j) = \sum_{k' \le k} \Delta f_\theta^{k'}(x_j)$$

- For the query token $j = N+1$, we set $y_{N+1} = 0$ because it's unknown; it just means "start prediction at 0".

✅ So each token stores:

- "what is the input x_j?"
- "how wrong am I right now on this point?" (residual)

This is why the slide earlier insisted on outputting Δf: it keeps the residual update clean.

Page 26: What are Keys and Queries?

The slide states: ECE590_Lecture10

$$\boxed{W_Q e_{j,k} = W_K e_{j,k} = \begin{pmatrix} x_j \\ 0 \end{pmatrix}}$$

Meaning

They deliberately choose W_Q and W_K to **throw away the residual** and keep only x_j.

So:

- **Keys** are x_i
- **Queries** are x_j

Therefore, the attention score becomes:

$$(W_K e_{i,k})^T (W_Q e_{j,k}) = x_i^T x_j$$

✅ That is exactly why inner products appear: attention score becomes the GD kernel $x_i^T x_j$.

Page 26: What are Values?

The slide states (for training tokens $j = 1, \ldots, N$): ECE590_Lecture10

$$\boxed{W_V e_{j,k} = \begin{pmatrix} 0 \\ y_j - \sum_{k'=1}^{k} \Delta f_\theta^{k'}(x_j) \end{pmatrix}}$$

Meaning

They design W_V to **throw away** x_j and keep only the residual.

So:

- **Value from token** i is essentially the residual $r_{i,k}$.

✅ This matches GD: updates use $(y_i - f_k(x_i))$.

Page 27: Put Q/K/V together into attention

Page 27 restates the projections (same as page 26) and then does the computation.

ECE590_Lecture10

The linear attention they compute is (up to their output scaling P):

$$P \sum_{i=1}^{N} W_V e_{i,k} \, (W_K e_{i,k})^T (W_Q e_{j,k})$$

Now substitute what each term is:

- $W_V e_{i,k} = \begin{pmatrix} 0 \\ r_{i,k} \end{pmatrix}$
- $(W_K e_{i,k})^T (W_Q e_{j,k}) = x_i^T x_j$

So attention output becomes:

$$P \sum_{i=1}^{N} \begin{pmatrix} 0 \\ r_{i,k} \end{pmatrix} (x_i^T x_j)$$

Now apply P. The slide earlier chose:

$$P = \begin{pmatrix} 0 & 0 \\ 0 & -\alpha \end{pmatrix}$$

So it multiplies the bottom scalar by $-\alpha$, giving:

$$\sum_{i=1}^{N} \begin{pmatrix} 0 \\ -\alpha r_{i,k} (x_i^T x_j) \end{pmatrix} = \begin{pmatrix} 0 \\ -\alpha \sum_{i=1}^{N} r_{i,k} x_i^T x_j \end{pmatrix}$$

Page 27 then names this bottom term as $-\Delta f_\theta^{k+1}(x_j)$: ECE590_Lecture10

$$\boxed{-\alpha \sum_{i=1}^{N} \left(y_i - \sum_{k' \le k} \Delta f_\theta^{k'}(x_i) \right) x_i^T x_j = -\Delta f_\theta^{k+1}(x_j)}$$

✅ So the attention output vector is:

$$\boxed{\text{attention output at token } j = \begin{pmatrix} 0 \\ -\Delta f_\theta^{k+1}(x_j) \end{pmatrix}}$$

This is the “differential output” idea: attention returns only the change.

How do we get the new embedding? (the skip connection)

This is page 28, but it's the immediate next line after page 27's result, so I'll say it:

Input:

$$e_{j,k} = \begin{pmatrix} x_j \\ r_{j,k} \end{pmatrix}$$

Attention output:

$$\begin{pmatrix} 0 \\ -\Delta f_\theta^{k+1}(x_j) \end{pmatrix}$$

Skip add:

$$e_{j,k+1} = \begin{pmatrix} x_j \\ r_{j,k} - \Delta f_\theta^{k+1}(x_j) \end{pmatrix} = \begin{pmatrix} x_j \\ y_j - \sum_{k' \le k+1} \Delta f_\theta^{k'}(x_j) \end{pmatrix}$$

✅ So each layer:

- keeps x_j unchanged
- updates the residual by subtracting the new correction

That is literally "doing GD on the function estimate" layer-by-layer.

The whole point of pages 26–27 in one sentence

> With a specific embedding format $e_{j,k} = (x_j, \text{residual})$ and carefully chosen W_Q, W_K, W_V, P, linear attention computes exactly the GD correction $\Delta f^{k+1}(x_j)$ using similarity $x_i^T x_j$ and residual values $y_i - f_k(x_i)$. ECE590_Lecture10

Quick "symbol dictionary" for these pages

- j: which token we are updating (could be training token or query)
- i: which context token contributes to the update
- k: layer index (also GD step index)
- $e_{j,k}$: token embedding at layer k, storing $(x_j, \text{residual})$
- W_Q, W_K: extract x_j only (so attention score becomes $x_i^T x_j$)
- W_V: extract residual only (so values are $y_i - f_k(x_i)$)
- P: scales by $-\alpha$ and puts update in the residual slot

- $\Delta f_\theta^{k+1}(x_j)$: the GD-style correction computed at layer $k+1$

If you want, next we can do either:

- a **tiny numeric run** through these exact formulas showing how residuals update from $k = 0 \to 1 \to 2$ using the *page 26–27* setup, or
- move forward to **page 30+** where they generalize to **multi-head + kernels** (nonlinear functions).

Which direction do you want?

Sources

ECE590_Lecture10.pdf
PDF

ChatGPT 5.2

Again, make it crystal clear, especially with each notation meaning, and step-by-step, don't just give me like 12 sections with 3 bullet points for each, I need details for each 12 sections if so

Thought for 49s

Yep — starting **page 33 ("Map to Feature Space")**, Lecture 10 is basically saying:

> "We already showed a Transformer *can implement gradient descent (GD) in its forward pass* for **linear** functions. Now we want the same idea for **nonlinear** functions — by turning the problem into a *linear* one in a bigger feature space (kernels)."

Below I'll go **page-by-page (33 → end)** and decode every symbol carefully.

Page 33 — "Map to Feature Space"

They introduce a **feature map**:

- Input: $x \in \mathbb{R}^d$
- Map it to: $\phi(x) \in \mathbb{R}^{d'}$ (they write d')

Key points:

- d' can be **different from** d, and can even be **infinite**.
- Then do **linear regression in feature space**: linear in $\phi(x)$, but nonlinear in x.

  ECE590_Lecture10

So the "nonlinearity" is pushed into $\phi(\cdot)$. The model becomes linear again *after* mapping.

Page 34 — Why this creates nonlinearity in original x-space

They define the model:

$$f_\theta(x) = w^T \phi(x) + b, \quad \theta = (w, b)$$

In **feature space**, it's a hyperplane (linear).

But when you look back in **original x-space**, the set of points that map onto a hyperplane in ϕ-space usually forms a **nonlinear surface** (curves, circles, blobs, etc.).

ECE590_Lecture10

So:

- linear in $\phi(x)$
- nonlinear in x

That's the whole trick.

Page 36 — "Inner Products in Feature Space" → the kernel trick

Now they define:

$$\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

This κ is the **kernel** (they call it a Mercer kernel). ECE590_Lecture10

Why does this matter?

Because if you can compute $\kappa(x_i, x_j)$ **directly** (without ever writing ϕ), then you can act *as if* you're working in that huge feature space.

They also write a standard kernel-machine form:

- represent a weight vector as a linear combination of feature vectors:

$$w = \sum_{m=1}^{M} \alpha_m \phi(x_m)$$

- then prediction is:

$$f_\theta(x) = \phi(x)^T w = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m)$$

So the "parameters" become $\{\alpha_m\}$ (and possibly $\{x_m\}$ depending on setup).

ECE590_Lecture10

Translation: the model output is a weighted sum of "similarities" between x and stored points x_m.

Page 37–38 — Kernel machines + examples

They summarize kernel regression / kernel machines:

$$f_\theta(x) = \sum_{m=1}^{M} \alpha_m \kappa(x, x_m) + b$$

And emphasize you often **don't explicitly define** ϕ — you just pick κ. ECE590_Lecture10

Then they list common kernels (linear, RBF, Laplacian, polynomial, cosine, etc.).

ECE590_Lecture10

Page 42 — Gradient descent, but in feature space → kernelized GD

This is the *big* math step.

They consider squared loss in feature space:

$$L(w) = \frac{1}{2}\sum_{i=1}^{N}\left(y_i - w^T\phi(x_i)\right)^2$$

GD update:

$$w_{k+1} = w_k + \alpha\sum_{i=1}^{N}\left(y_i - w_k^T\phi(x_i)\right)\phi(x_i)$$

Now evaluate the function at a point x_j:

$$f_{w_{k+1}}(x_j) = w_{k+1}^T\phi(x_j)$$

Substitute the GD update into this and you get:

$$f_{w_{k+1}}(x_j) = f_{w_k}(x_j) + \alpha\sum_{i=1}^{N}\left(y_i - f_{w_k}(x_i)\right)\kappa(x_i, x_j)$$

This is exactly what the slide shows. ECE590_Lecture10

Intuition (super important)

At step $k+1$, the function value at x_j is updated by:

- taking the residuals $(y_i - f_k(x_i))$
- weighting each residual by similarity $\kappa(x_i, x_j)$
- summing them up

That looks *extremely* like attention.

Page 43 — "From Linear Attention to Kernel Attention"

They name the increment:

$$\Delta f_{w_{k+1}}(x_j) = \alpha\sum_{i=1}^{N}(y_i - f_{w_k}(x_i))\kappa(x_i, x_j)$$

And they say: initialize $w_0 = 0 \Rightarrow f_{w_0}(x) = 0$. ECE590_Lecture10

This parallels what they did earlier for "linear attention implements GD".

Page 44–45 — Adding a bias term needs *another* kind of attention head

When you include bias b:

$$L(w, b) = \frac{1}{2} \sum_{i=1}^{N} (y_i - w^T \phi(x_i) - b)^2$$

GD gives **two** updates:

- w-update looks kernel-weighted (like above)
- b-update is:

$$b_{k+1} = b_k + \alpha \sum_{i=1}^{N} (y_i - f_k(x_i))$$

Notice: the b update is like the same residual sum, but with **no dependence on** x_j (no kernel term).

So the function update becomes:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i)) \kappa(x_i, x_j) + \alpha \sum_i (y_i - f_k(x_i))$$

That second piece is a **constant-attention** piece. ECE590_Lecture10

Page 45 conclusion:

You need **two attention heads**:

1. a "kernel" head using $\kappa(x_i, x_j)$
2. a "constant" head that always returns 1 (so it just sums residuals)

That's literally what "Two Attention Heads" says. ECE590_Lecture10

✅ This answers one of your earlier confusions: **multi-head attention isn't just for "more power" — here it's needed to represent different update terms (kernel term + bias term).**

Pages 46–50 — "Transformer Design (1/5 … 5/5)" → how W_Q, W_K create those heads

They pick a specific kernel: **RBF**

$$\kappa(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

and encode each token as $e_i = (x_i, y_i)^T$ (and query token $e_{N+1} = (x_{N+1}, 0)^T$).

ECE590_Lecture10

Head 1: make the "kernel attention" depend only on x, not y

They set:

$$W_Q^{(1)} = W_K^{(1)} = \begin{pmatrix} I_d & 0 \\ 0 & 0 \end{pmatrix}$$

So when you multiply $e_i = (x_i, y_i)$, you get $(x_i, 0)$. That means the kernel becomes:

$$\kappa(W_K^{(1)} e_i,\, W_Q^{(1)} e_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right)$$

Exactly RBF on the x's. ECE590_Lecture10

Head 2: constant attention = 1

They set:

$$W_Q^{(2)} = W_K^{(2)} = 0$$

Then $W_K^{(2)} e_i = 0$ and $W_Q^{(2)} e_j = 0$, so $\|0 - 0\| = 0$, and RBF kernel becomes $\exp(0) = 1$. Boom: constant attention. ECE590_Lecture10

Values and output matrices (how residuals get multiplied and scaled)

The later "Transformer design" slide shows they use the **same** W_V and W_O forms for both heads:

$$W_V^{(1)} = W_V^{(2)} = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad W_O^{(1)} = W_O^{(2)} = \begin{pmatrix} 0 & 0 \\ 0 & -\alpha \end{pmatrix}$$

(Shown on page 50.) ECE590_Lecture10

Meaning:

- W_V is picking out the **scalar residual-like component** (the "y channel" in their (x, y) stacked encoding).
- W_O scales it by $-\alpha$ so the update matches GD step size.

So W_V is literally "what information we pass along from each context example into the update."

Page 51 — Softmax kernel + why multi-head matters

They admit something important:

- GD theory wants a **Mercer kernel** (valid inner-product kernel).
- The Transformer's actual attention uses **softmax** over dot-products, which doesn't neatly fit the Mercer-kernel picture.
- But there are Mercer kernels "close" to it, and the kernel-machine view still helps explain why *nonlinear attention* and *multi-head mixtures* are useful.
  ECE590_Lecture10

Also they explicitly say:

- skip connection becomes clear: it's needed to implement the "accumulate updates across layers" behavior (like iterative GD). ECE590_Lecture10

Page 52 — They paste "Attention Is All You Need" architecture, including FFN

This page reminds you the real Transformer layer has:

1. multi-head self-attention
2. **position-wise feed-forward network (FFN)**
   with residual + LayerNorm around each.

They quote: "simple, position-wise fully connected feed-forward network" and "LayerNorm(x + Sublayer(x))". ECE590_Lecture10

Now: where does the FFN fit in *this lecture's story*?

Lecture 10 mostly analyzed **attention + skip** as "GD-like update." But in actual Transformers, FFN is crucial.

A clean way to understand FFN here:

1) Attention mixes information across tokens

It computes weighted sums across positions (context-dependent mixing).

2) FFN transforms each token independently

It's "point-wise": same MLP applied to each position.

In the kernel / feature-space language introduced on page 33:

- attention is acting like a **kernel-weighted update / regression step** based on similarities.
- FFN is like learning a **new feature map** (a learned nonlinear $\phi(\cdot)$-like transform) *between* those mixing steps.

So a very exam-friendly one-liner:

> **Attention** = "contextual aggregation / kernel mixing across tokens"
> **FFN** = "per-token nonlinear feature transform (increases representational power)"
> **Stacking layers** = "iterate: mix → nonlinearly remap → mix → …"

And multi-head helps because you can combine different "kernels / update terms" simultaneously (as they explicitly showed with kernel-head + constant-head).

ECE590_Lecture10

Quick checkpoint: what page are we on now?

We have now covered (in order):

- **p33–35:** feature mapping idea $\phi(x)$ → nonlinear in x but linear in $\phi(x)$
  ECE590_Lecture10
- **p36–41:** kernels $\kappa(x_i, x_j) = \phi(x_i)^T \phi(x_j)$, kernel machines, RBF example, RKHS
  ECE590_Lecture10
- **p42–45:** GD in feature space → kernelized functional GD update; bias needs a constant head ECE590_Lecture10

- **p46–51:** explicit construction of two heads via W_Q, W_K for RBF-kernel head + constant head; notes about softmax kernel ECE590_Lecture10 ECE590_Lecture10
- **p52:** back to canonical Transformer layer + FFN ECE590_Lecture10

If you want, next I can do a **tiny numeric toy run** of the *kernelized GD update* (with 2 context points + 1 query) and show how it corresponds to "attention weights = similarities," and then how adding the constant head mimics the bias update.

Sources

what does the boom mean in your head 2 constant attention = 1? Why boom?

Great catch — and thank you for stopping me there. Let's **slow this down and remove the "boom" drama**. Nothing magical is happening; it's just a **deterministic consequence of the kernel definition**.

I'll explain **exactly** why *Head 2 gives constant attention = 1*, step by step, with no hype.

Where this comes from (context recap)

On pages **44–45**, Lecture 10 is handling **bias** b in kernel regression.

They showed that when you include a bias term, gradient descent produces **two update terms**:

$$f_{k+1}(x_j) = f_k(x_j) + \underbrace{\alpha \sum_i (y_i - f_k(x_i))\kappa(x_i, x_j)}_{\text{kernel-dependent term}} + \underbrace{\alpha \sum_i (y_i - f_k(x_i))}_{\text{bias term}}$$

Key observation:

- The **bias term does NOT depend on** x_j.
- It treats *every query the same*.

So the Transformer needs:

- one attention head whose weights depend on similarity ($\kappa(x_i, x_j)$)
- one attention head whose weights are **the same for all** i, j

That second head is what they call **constant attention**.

Now: what exactly is "Head 2: constant attention = 1"?

Step 1: Recall the kernel they chose

They use an **RBF (Gaussian) kernel**:

$$\boxed{\kappa(u, v) = \exp\left(-\frac{\|u - v\|^2}{2\sigma^2}\right)}$$

This kernel has one very important property:

$$\kappa(u, u) = \exp(0) = 1$$

No approximation. No trick. Just math.

Step 2: What do $W_Q^{(2)}$ and $W_K^{(2)}$ do?

On the slide, they **define**:

$$\boxed{W_Q^{(2)} = W_K^{(2)} = 0}$$

This means:

For *any* token embedding e_i,

$$W_K^{(2)} e_i = 0, \quad W_Q^{(2)} e_j = 0$$

So **every key and every query becomes the same vector: the zero vector**.

Step 3: Plug this into the kernel

Now compute the attention weight between *any* pair i, j:

$$\kappa\big(W_K^{(2)} e_i,\ W_Q^{(2)} e_j\big) = \kappa(0,0)$$

Using the RBF formula:

$$\kappa(0,0) = \exp\left(-\frac{\|0-0\|^2}{2\sigma^2}\right) = \exp(0) = \boxed{1}$$

This is true for **every** i, j.

What this means (no hype, just interpretation)

- Every token attends to every other token **with equal weight**
- There is **no dependence on input features**
- The attention weight is a **constant scalar**

That is all "constant attention = 1" means.

Why this is exactly what we need for the bias update

Recall the bias GD term:

$$\alpha \sum_i (y_i - f_k(x_i))$$

Notice:

- No x_j
- No similarity
- Just a sum over residuals

Now look at what attention with constant weights does:

$$\sum_i \underbrace{1}_{\text{attention weight}} \cdot \text{Value}_i = \sum_i \text{Value}_i$$

And since W_V extracts the residual:

$$\text{Value}_i = y_i - f_k(x_i)$$

We get **exactly** the bias update term.

So let's restate it cleanly (replace "boom" with logic)

> By choosing $W_Q = W_K = 0$, every query and key maps to the same point in feature space.
> Under the RBF kernel, this yields a constant similarity of 1 between all token pairs, producing attention weights that are independent of inputs and implement the bias update term in gradient descent.

That's it. No magic. No shortcut. Just a controlled design choice.

Why I said "boom" (and why you were right to question it)

I said "boom" because **this is the conceptual punchline** of that slide:

- Multi-head attention is *not just extra capacity*
- It is *structurally necessary* to represent different GD update components
  - kernel-dependent term
  - bias term

But you were 100% right to stop and ask **why**.

If you want next:

- we can do a **tiny numeric example** showing kernel-head + constant-head working together
- or connect this directly to **softmax attention**
- or prep **exam-style explanations** for this exact slide

Just tell me the next step.

ECE590_Lecture11.pdf
PDF

read the lec 11 slides carefully. Is it simply a repeat of lec 10? Anything that I need to specifically careful about in lec 11

Short answer first (so you don't over- or under-study):

> **Lecture 11 is *not* just a repeat of Lecture 10.**
> It **re-derives the same core idea**, but **changes the emphasis** and introduces **exam-relevant clarifications** that *do not appear explicitly* in Lecture 10.

Below I'll be very concrete: **what is repeated**, **what is new**, and **what you must be careful about**.

1️⃣ What Lecture 11 *repeats* from Lecture 10

Yes — a large fraction is intentionally repeated, but with a different pedagogical goal.

Repeated core thesis

Lecture 11 again shows:

> **Attention + skip connection can implement gradient descent in the forward pass**, enabling few-shot / in-context learning **without parameter updates**.

This appears explicitly on:

- **Slides 2–3** (isolate attention + skip)
- **Slides 5–6** (linear functions, GD update)
- **Slides 7–14** (step-by-step construction with Φ, W_V, P)

So if you understand **Lecture 10 pages 17–27**, Lecture 11 pages **2–15** should feel familiar.

That repetition is intentional.

2️⃣ What Lecture 11 adds that Lecture 10 does *not* emphasize

This is the important part.

(A) Lecture 11 explicitly reframes this as meta-learning

On **slides 4–5**, Lecture 11 introduces:

$$\{C_m\}_{m=1}^{M}, \quad C_m = \{(x_i^{(m)}, y_i^{(m)})\}_{i=1}^{N+1}$$

And stresses:

- Each dataset has its **own latent** w_m
- The Transformer is trained across many such tasks
- At test time, it sees a *new task* C_{M+1}

Lecture 10 *implied* this.
Lecture 11 **states it clearly**.

📌 **Be careful:**
In exam answers, you should say **"meta-model"** or **"learning to learn"** when describing this setting. Lecture 11 makes that framing explicit.

(B) Lecture 11 is much clearer about layer = GD step

Lecture 11 repeatedly emphasizes:

> **Each Transformer layer implements one step of gradient descent.**

This is explicitly stated on:

- Slide 6
- Slide 16 ("Each layer of Transformer implements one step of GD")

Lecture 10 showed it mathematically;
Lecture 11 **wants you to say it in words**.

📌 **Exam-safe phrasing**:

> "The depth of the Transformer corresponds to the number of gradient-descent-like updates performed in function space."

(C) Lecture 11 highlights preconditioned gradient descent

This is new emphasis.

On **Slide 18**, Lecture 11 says:

> "If we train the linear Transformer parameters based on multiple contextual examples, it can do better than GD: *preconditioned GD*."

Meaning:

- The Transformer doesn't have to implement *vanilla* GD
- Learned W_Q, W_K, W_V, P can implement:
  - different step sizes
  - different conditioning directions
  - data-dependent updates

📌 **This is subtle and exam-relevant**:

> The Transformer learns *how* to perform GD, not just to copy it.

Lecture 10 does not highlight this as explicitly.

3️⃣ How Lecture 11 reorganizes the kernel story

Lecture 11 reorganizes Lecture 10's kernel discussion into a **clean three-stage story**.

Stage 1 — Linear functions (Slides 6–18)

Same as Lecture 10:

- Linear $f(x) = w^T x$
- Linear attention
- Skip connection accumulates updates

Stage 2 — Nonlinear via feature map $\phi(x)$ (Slides 19–22)

This is where Lecture 11 restarts the kernel story more slowly:

- Map $x \mapsto \phi(x)$
- Linear in feature space ⇒ nonlinear in input space

This is pedagogically clearer than Lecture 10.

Stage 3 — Kernel machines & two attention heads (Slides 23–38)

Same conclusion as Lecture 10:

- Kernel attention head for $\kappa(x_i, x_j)$
- Constant attention head for bias
- Multi-head attention is *structurally necessary*

But Lecture 11 makes one thing **very explicit**:

> **Multi-head attention is not "just more expressiveness" — it corresponds to multiple update terms in the learning rule.**

That sentence is *implicitly* in Lecture 10, but **explicit** in Lecture 11.

4️⃣ What you must be careful about in Lecture 11

Here are the **actual pitfalls** students fall into.

⚠️ Pitfall 1: Thinking Lecture 11 introduces a new model

It does **not**.

Lecture 11 is:

- a reinterpretation
- a reframing
- a pedagogical consolidation

If asked:

> "Is Lecture 11 proposing a new architecture?"

Correct answer:

> **No — it reinterprets the Transformer architecture through the lens of meta-learning and gradient descent.**

⚠️ Pitfall 2: Confusing "linear attention" with "softmax attention"

Lecture 11 explicitly contrasts:

- linear attention (for theory)

- softmax attention (used in practice)

On **Slide 16**, they write both formulas side by side.

Be careful:

- Theory uses linear attention
- Practice uses softmax
- Kernel view is an *interpretive lens*, not a literal implementation claim

📌 Exam phrasing:

> "Softmax attention is not a Mercer kernel, but can be interpreted as approximating kernel-like similarity."

⚠️ Pitfall 3: Ignoring the FFN question at the end

The last slide asks explicitly:

> **"What about the FF block?"**

This is a hint:

- Everything so far explained **attention + skip**
- FFN is not analyzed here
- You are *not* expected to derive FFN mathematically like attention

Safe interpretation:

> FFN provides per-token nonlinear feature transformation that complements attention-based context mixing.

If you over-theorize FFN here, you may lose points.

5️⃣ Final exam-oriented comparison

Is Lecture 11 just a repeat?

No.

It is a **reframing + clarification** lecture.

What Lecture 10 does best

- Deep math
- Exact constructions
- Proof-of-existence

What Lecture 11 does best

- Meta-learning framing
- Layer-as-GD-step intuition
- Role of multi-head attention
- Cleaner narrative for nonlinear extension

6️⃣ What I would memorize from Lecture 11 (high-yield)

If you want a **minimal exam checklist**:

1. "Each Transformer layer ≈ one GD step in function space"
2. "Learning happens in activations, not parameters"
3. "Multi-head attention corresponds to multiple update terms (kernel + bias)"
4. "Transformer training learns *how* to learn (meta-learning)"
5. "Kernel view explains nonlinear in-context learning"

If you want, next I can:

- compare **Lecture 10 vs Lecture 11 slide-by-slide**
- give you **model answers** for likely exam questions
- explain **how FFN fits into this story without overclaiming**
- help you write a **2–3 paragraph exam explanation** of the whole theory

Just tell me how you want to finish prepping.

what is preconditioned GD? what is meta-model or learning to learn? what is a few-shot learning?

Great — these three terms are **conceptual pillars** of Lec 10–11, and they're often used loosely. I'll explain them **from first principles**, then tie them **directly back to Transformers**, and finally give you **exam-ready phrasing**.

I'll do this **slowly and concretely**, not buzzword-y.

1️⃣ What is Preconditioned Gradient Descent (PGD)?

Start from plain Gradient Descent (GD)

For minimizing a loss $L(w)$, vanilla GD is:

$$w_{k+1} = w_k - \alpha \nabla L(w_k)$$

Key properties:

- Same step size in **all directions**
- Treats all coordinates equally
- Can be **slow** or **zig-zaggy** if the problem is ill-conditioned

---

What does "preconditioned" mean?

Preconditioning means:

> Before taking a gradient step, **transform the gradient** using a matrix.

Mathematically:

$$\boxed{w_{k+1} = w_k - \alpha\, M\, \nabla L(w_k)}$$

where:

- M is a **preconditioning matrix**
- M is usually symmetric positive definite

Intuition (very important)

- GD assumes the loss landscape is "round"
- Real problems are usually "stretched ellipses"
- Preconditioning **rescales and rotates** the space so GD behaves better

Think:

- Newton's method ≈ special preconditioned GD
- Adam / RMSProp ≈ adaptive diagonal preconditioners

Where this appears in Lec 11

Lecture 11 says (slide ~18):

> "If we train the linear Transformer parameters based on multiple contextual examples, it can do better than GD: **preconditioned GD**."

Meaning:

- In Lec 10, we **hand-designed** W_Q, W_K, W_V, P to match *vanilla* GD
- In practice, the Transformer **learns** these matrices from data
- Learned matrices ≠ identity
- Therefore the Transformer implements **preconditioned GD**

Translation into Transformer language

GD concept	Transformer analogue
Gradient	Residuals extracted by W_V
Step size α	Learned output matrix P
Preconditioner M	Learned projections W_Q, W_K, W_V
Conditioning on data	Attention weights

So the Transformer isn't just copying GD — it's learning **how to optimize better than GD**.

One-sentence exam answer

> **Preconditioned gradient descent** is gradient descent where the update direction is transformed by a learned matrix; in Transformers, learned attention and value projections act as a data-dependent preconditioner, enabling faster and more adaptive in-context optimization.

2️⃣ What is a Meta-Model / Learning to Learn?

This is the **most important conceptual shift**.

Ordinary learning (what you already know)

- Train a model on a dataset
- Learn parameters w
- Deploy model
- Parameters stay fixed

Example:

> Train linear regression → get w → use forever

Meta-learning ("learning to learn")

Now imagine this instead:

- You are given **many small datasets**
- Each dataset comes from a **different task**
- You want one system that can **adapt quickly** to a new task

---

Formal setup (used in Lec 11)

You have tasks $m = 1, \ldots, M$:

$$C_m = \{(x_i^{(m)}, y_i^{(m)})\}_{i=1}^{N+1}$$

Each task has:

- its own latent function f_{θ_m}
- its own hidden parameters (e.g., w_m)

What is the meta-model?

The **meta-model** is the Transformer.

- It is trained across many tasks $\{C_m\}$
- It does **not** learn w_m directly
- It learns an **algorithm** for inferring w_m from examples

So:

> **The Transformer is not the task model — it is the learner of task models.**

Why "learning to learn"?

Because the Transformer learns:

- how to update predictions
- how to use context
- how to generalize from a few examples

Instead of learning:

- the final function itself

In Lec 10–11 language

Thing	Meaning
w_m	Task-specific parameter (never stored explicitly)
Transformer weights	Meta-parameters (shared across tasks)
Forward pass	Executes the learned learning algorithm

One-sentence exam answer

> A **meta-model** is trained across many tasks to learn an inference or optimization procedure; in Transformers, the learned attention mechanism acts as a task-agnostic learning algorithm that adapts to new tasks during inference.

3️⃣ What is Few-Shot Learning?

This one is simpler, but crucial.

Definition (clean)

> **Few-shot learning** means making accurate predictions using only a **small number of labeled examples** (e.g., 1–10).

Contrast:

- Traditional ML: thousands or millions of examples
- Few-shot: extremely limited data

How this fits into Lec 10–11

In each task C_m:

- You see only N examples
- Then you are asked to predict y_{N+1}

The Transformer must:

- infer the underlying function
- using **very little data**
- in a single forward pass

That is exactly **few-shot learning**.

Why Transformers can do this

Because:

- Attention compares the query to examples

- Skip connections accumulate updates
- Depth ≈ number of update steps

So the model performs **inference-time learning**.

Important distinction (often tested)

Concept	Meaning
Few-shot learning	Ability to learn from few examples
Meta-learning	Training paradigm that enables few-shot learning
In-context learning	How Transformers implement few-shot learning

---

One-sentence exam answer

> **Few-shot learning** refers to learning a task from a small number of examples; Transformers achieve this via in-context learning, where attention and skip connections perform task adaptation during the forward pass.

4️⃣ How all three concepts fit together (VERY IMPORTANT)

Here is the **full picture** Lec 10–11 wants you to see:

1. The Transformer is trained as a **meta-model**
2. It learns **how to update predictions** (learning to learn)
3. At inference time:
   - it receives a few examples (few-shot)
   - it performs preconditioned GD-like updates
   - inside its activations
4. No parameters change

One final killer summary (memorize this)

> Transformers are meta-learners trained across tasks to learn a preconditioned gradient-descent-like algorithm, enabling few-shot learning through in-context adaptation during the forward pass without parameter updates.

If you want next, I can:

- turn this into **exam-perfect paragraphs**
- compare this to **MAML** (classic meta-learning)
- connect this to **GPT in-context learning**
- give you **diagrammatic intuition**

Just tell me.

did the slides mention how to learn WQ,WK,WV?

Short, precise answer first — then I'll unpack it carefully:

> **No. The slides do *not* derive how to learn W_Q, W_K, W_V mathematically.**
> They **assume they are learned by standard training (backpropagation)** and focus instead on **what *can be represented* once they are learned**.

This distinction is *very important* for exams.

1️⃣ What the slides actually do (and do NOT do)

What they do

- **Construct specific W_Q, W_K, W_V, P** by hand
  → to **prove existence**:

  > "There exists a Transformer configuration that implements (preconditioned) GD."

- Show that **if** these matrices have certain forms, then:
  - attention = kernel similarity

- values = residuals
- skip connections = accumulation of updates

This is a **representational proof**, not a training algorithm.

What they do NOT do

- They do **not**:
  - give a loss function for learning W_Q, W_K, W_V
  - show gradient updates for these matrices
  - analyze convergence of training
  - claim Transformers explicitly optimize GD during training

So if you're wondering:

> "Where is the equation for learning W_Q?"

Answer: **It's not there. On purpose.**

2️⃣ So how *are* W_Q, W_K, W_V learned in reality?

This part is **implicit**, but standard.

In practice (and what the slides assume you know)

- The Transformer is trained **end-to-end with backpropagation**
- Objective: prediction loss over many tasks C_m
- Gradients flow through:
  - attention
  - softmax
  - FFN
- This updates:

$$W_Q,\ W_K,\ W_V,\ W_O,\ \text{FFN weights}, \ldots$$

Nothing special or new here.

The novelty is **how those learned weights are *interpreted***.

3️⃣ Why the slides intentionally avoid "learning equations" for W_Q, W_K, W_V

This is subtle but exam-relevant.

Reason 1: The goal is *interpretation*, not training

Lecture 10–11 ask:

> "What algorithm does a trained Transformer *implement* at inference time?"

Not:

> "How do we train Transformers?"

So the focus is on:

- **forward-pass behavior**
- **functional updates**
- **meta-learning interpretation**

Reason 2: The constructed matrices are *not claimed to be optimal*

The hand-designed W_Q, W_K, W_V:

- are **existence proofs**
- are **not** how real models look
- are **not** what SGD necessarily finds

The slides even hint at this when they say:

> "If we train the linear Transformer parameters … it can do *better than GD*: preconditioned GD."

That's an admission that:

- learned matrices will generally be *different*
- and often *better* than the hand-designed ones

4️⃣ How learning W_Q, W_K, W_V fits into the meta-learning story

Here's the clean conceptual split:

Two levels of learning

🔵 **Outer loop (training time)**

- Learn W_Q, W_K, W_V
- Via SGD / Adam
- Across many tasks C_m
- Objective: minimize expected task loss

This is **meta-learning**.

🟢 **Inner loop (inference time)**

- No parameter updates
- Activations evolve layer by layer
- Attention + skip connections perform GD-like updates

This is **in-context learning**.

5️⃣ Exam-safe answers (use these verbatim if needed)

Short answer

> The slides do not derive how to learn W_Q, W_K, W_V; they assume these are learned via standard backpropagation and focus on showing that, once learned, the Transformer can implement gradient-descent-like updates during inference.

Slightly longer (very safe)

> Lecture 10–11 provide an existence and interpretation result: they show that there exist choices of W_Q, W_K, W_V for which attention implements (preconditioned) gradient descent. The slides do not analyze how these matrices are learned; in practice they are optimized via end-to-end training across tasks using backpropagation.

If the exam asks “why is this okay?”

> Because the goal of the lecture is not to propose a new training algorithm, but to explain the emergent behavior of trained Transformers as meta-learners performing

in-context optimization.

6️⃣ One last sanity check (to reassure you)

If the slides *had* tried to derive learning rules for W_Q, W_K, W_V, they would:

- explode in complexity
- distract from the main idea
- overlap with earlier deep learning lectures

So their omission is **intentional and correct**.

If you want, next I can:

- help you write a **perfect exam paragraph** explaining this distinction
- contrast this with **MAML**, which *does* explicitly optimize inner-loop learning
- explain **why SGD training naturally leads to GD-like inference behavior**

Just tell me.

Exam 1 with GPT4o Solu...
PDF

any questions here in this exam set is related to the transformer we have learned so far

Great question — and yes, **this exam is very intentionally aligned with Lectures 10–11**. It does **not** ask about Transformers explicitly by name, but several questions are **direct instantiations of the Transformer-as-GD / in-context learning view** you just learned.

I'll go **problem by problem**, tell you **whether it's related**, **how**, and **what you should be careful about**.

I'll base this directly on *Exam 1: Large Language Models* Exam 1 with GPT4o Solutions .

🔷 Problem 1 — YES, this is Transformer-related (very strongly)

What it looks like on the surface

Problem 1 talks about:

- a **softmax model**
- a latent vector f
- updating f by **gradient descent**
- a **feedforward network with a skip connection**

No mention of "Transformer" explicitly.

What it *actually* is

This problem is a **minimal abstraction of one Transformer block** operating on a latent representation.

Let's map it explicitly.

Exam notation	Transformer interpretation
f	latent token representation
softmax over $w_c^T f$	softmax classifier / language head
gradient descent on f	*inference-time update* (in-context learning)
$g_\theta(f_0, w_y)$	feedforward sublayer
skip connection $f_0 + f_{\text{out}}$	residual connection
alignment with gradient	one GD-like update

The key idea being tested

Parts (c)–(d) are **exactly Lecture 11**:

> A feedforward block + skip connection can implement **one step of gradient descent** on a latent representation.

This mirrors the lecture claim:

> "Each Transformer layer performs one GD-like update in function space."

What to be careful about (exam trap)

- You are **not** supposed to say "this is a Transformer"
- You are supposed to say:
  - *"the skip connection allows the model to accumulate an update"*
  - *"the feedforward output aligns with the gradient"*

If you say "attention" here, you'd actually be **wrong** — this problem isolates the **FFN + skip** mechanism.

Verdict for Problem 1

✅ **Directly tests Lecture 10–11 ideas**, especially:

- skip connections = GD accumulation
- learning happens in activations
- alignment with gradient = optimization step

🔷 Problem 2 — YES, this is *even more directly* Lecture 10–11

What it looks like

Problem 2 discusses:

- contextual data (x_i, y_i)
- a query x_{N+1}
- a latent function $f(x)$
- a feature map $\phi(x)$
- a kernel $\kappa(x_i, x_j)$
- gradient descent in function space

This is **almost a copy** of Lecture 10′s math.

What it *actually* is

This problem is testing whether you understand:

> **Kernelized gradient descent = attention-based function update**

This is the core of:

- Lecture 10 (functional GD → attention)
- Lecture 11 (kernel view + meta-learning)

Exact correspondence to lectures

Exam object	Lecture meaning
$f(x) = A\phi(x)$	linear model in feature space
rows a_m	per-output-dimension parameters
GD on A	learning task-specific function
kernel $\kappa(x_i, x_j)$	attention similarity
functional update (12)	attention-weighted update
initialization $f^{(0)} = 0$	zero initial representation

Equation (12) in the exam:

$$f^{(1)}(x_j) = f^{(0)}(x_j) + \alpha \sum_i [w_{y_i} - \mathbb{E}(w) \mid f^{(0)}(x_i)]\kappa(x_i, x_j)$$

This is **exactly** the same update you saw in:

- Lecture 10 (linear + kernel GD)
- Lecture 11 (kernel attention head)

What to be careful about (very important)

- Do **not** confuse *training-time GD on* A with *inference-time GD on* f
- The exam wants you to recognize:

  > learning in **function space**, not parameter space

- When they set $f^{(0)}(x) = 0$, that corresponds to:
  - uniform softmax
  - average embedding
  - first attention layer behavior (they literally say this at the end)

Verdict for Problem 2

✅ **This is Lecture 10–11 almost line-by-line**, especially:

- kernel attention
- functional gradient descent
- few-shot / contextual inference

🔷 "Food for thought" section — Explicit Transformer connection

The last page of the exam literally says:

> "This looks very much like the first attention layer of the Transformer…"

That is the exam **confirming** the connection to the lectures Exam 1 with GPT4o Solutions .

So yes — the instructors *expect* you to see the Transformer link, even if the question itself doesn't say "Transformer".

✅ Final summary (what to take into the exam)

Are there Transformer-related questions?

Yes — Problems 1 and 2 are both Transformer-related.

Are they asking about architecture details?

❌ No (no Q/K/V matrices explicitly)

Are they asking about behavior?

✅ Yes:

- skip connections
- gradient descent in representations
- kernel-weighted updates
- learning without parameter updates

🔑 One killer sentence you should be ready to say

> These problems analyze how feedforward networks, attention-like kernel weighting, and skip connections can implement gradient-descent-style updates in latent function space, which is the theoretical basis for in-context learning in Transformers.

If you want, next I can:

- rewrite **Problem 1 or 2** explicitly as “this is a Transformer layer”
- give you **exam-style short answers** connecting them to Lec 10–11
- help you anticipate **likely exam variations**

Just tell me how you want to finish prep.

Sources

ECE590_Lecture12.pdf
PDF

what is bert

Great timing — **BERT is exactly where Lecture 12 “branches” from Lectures 10–11**, so let’s lock this down cleanly and calmly.

I’ll explain **what BERT is**, **why it exists**, **how it differs from GPT**, and **what the slides want you to remember for exams**, explicitly tying everything back to **Lecture 12**

ECE590_Lecture12

1. What is BERT? (one-sentence answer)

BERT = **B**idirectional **E**ncoder **R**epresentations from **T**ransformers

→ a Transformer model **pretrained to understand language**, not to generate it.

Key phrase from the slides:

> *"Bidirectional Encoder Representations from Transformers"* ECE590_Lecture12

---

2. Why was BERT introduced?

Lecture 12 explains this motivation very explicitly.

Problem with GPT-style models (Lectures 10–11)

GPT-style models:

- predict the **next word**
- use **masked (causal) attention**
- only look **left → right**

This makes them:

- excellent for **generation**
- less ideal for **understanding tasks**

From the slide:

> "The heart of GPT-style pre-training is based on predicting the next word"
> ECE590_Lecture12

Key insight behind BERT

If we **change the pretraining task**, we can:

- remove causal masking
- allow **bidirectional attention**
- learn **richer representations**

That's what BERT does.

3. What architecture does BERT use?

Important:

BERT is **not a new architecture**.

It uses:

- the **same Transformer blocks**
- the same **attention + FFN + skip + norm**
- the same **Q, K, V mechanism**

What changes is **how it is trained**, not how it is built.

Lecture 12 makes this clear by reusing the same diagrams and blocks before introducing BERT ECE590_Lecture12 .

---

4. The two key pretraining tasks in BERT

(A) Masked Language Modeling (MLM)

Instead of predicting the *next* word:

➡️ **Randomly mask words anywhere in the sentence**
➡️ Predict the **missing word** using both left and right context

From slides:

- 15% of tokens are selected
- Of those:
  - 80% replaced with `[MASK]`
  - 10% replaced with a random word
  - 10% unchanged

Slides pages 59–62 explain this explicitly ECE590_Lecture12 .

📌 **Why this matters**

- Attention is **fully bidirectional**
- Each token can attend to **all other tokens**

This is the biggest conceptual difference from GPT.

(B) Next Sentence Prediction (NSP)

A **classification task built from unlabeled text**:

Given two sentences:

- Are they **actually consecutive** in the corpus?
- Or randomly paired?

From the slides:

> "Model learns to predict whether two consecutive given sentences actually followed each other" ECE590_Lecture12

📌 This forces BERT to learn:

- discourse structure
- sentence-level semantics

---

5. The [CLS] token (VERY exam-relevant)

BERT introduces a **special learned token**:

```
[CLS] sentence A [SEP] sentence B [SEP]
```

From Lecture 12:

Copy code

> "CLS is a special token identifying part of sequence used for classification tasks" ECE590_Lecture12

What happens to [CLS]?

- It flows through all Transformer layers
- Its **final vector** is treated as:

  a summary of the entire input

Used for:

- classification
- sentiment analysis
- multiple-choice
- entailment

This is **different from GPT**, which uses the *last token*.

---

6. GPT vs BERT (core exam comparison)

Straight from Lecture 12 slides ECE590_Lecture12 :

Aspect	GPT	BERT
Attention	Causal (masked)	Bidirectional
Pretraining	Next-word prediction	Masked-word prediction
Direction	Left → Right	Left ↔ Right
Best at	Generation	Understanding
Special token	None	`[CLS]`
Sentence-level task	No	NSP

---

7. How BERT is used after pretraining

After pretraining:

- add a **linear layer**
- fine-tune on downstream tasks

Exactly the same fine-tuning idea you saw earlier — but now:

- features are **stronger**
- less labeled data needed

Slides 63–66 show this pipeline ECE590_Lecture12 .

---

8. How this connects back to Lectures 10–11

This is the big conceptual loop closure.

Lecture	Core idea
Lec 10	Attention ≈ kernelized functional update
Lec 11	Transformer layers ≈ GD steps in representation space
Lec 12	BERT changes **training objective**, not the update mechanism

So:

- **BERT still "updates representations layer by layer"**
- but learns representations useful for **classification**, not generation

---

9. One exam-ready explanation (memorize this)

> BERT uses the same Transformer architecture as GPT, but changes the pretraining objective. Instead of predicting the next word using masked attention, BERT predicts randomly masked words using bidirectional attention and learns sentence-level structure via next-sentence prediction. This produces representations better suited for understanding and classification tasks rather than generation.

---

10. If you want, next we can:

- draw a **side-by-side GPT vs BERT computation**
- explain **why BERT is bad at generation**
- connect BERT back to **kernel attention and GD**
- practice **exam-style short answers**

Just tell me what you want to tackle next — and don't worry, you're *not* cooked 🙂

Sources

$$x_{\parallel} \;=\; proj_w(x) \;=\; \frac{\langle x, w\rangle}{\langle w, w\rangle}\, w.$$

what does the <> notation mean

Great question — this is **basic but foundational**, and once it clicks, a lot of ML / linear algebra notation becomes much clearer.

What does the ⟨·,·⟩ notation mean?

$$\boxed{\langle x, w\rangle \text{ means the **inner product** of vectors } x \text{ and } w}$$

In **standard Euclidean space** ($\mathbb{R}^n$), this is just the **dot product**.

Concretely: what is $\langle x, w\rangle$?

If

$$x = (x_1, x_2, \ldots, x_n), \quad w = (w_1, w_2, \ldots, w_n),$$

then

$$\boxed{\langle x, w\rangle = x_1 w_1 + x_2 w_2 + \cdots + x_n w_n}$$

In matrix notation:

$$\langle x, w\rangle = x^\top w$$

Why do we use ⟨·,·⟩ instead of $x^\top w$?

Because ⟨·,·⟩ is **more general**.

Dot product is a *special case* of an inner product

In general math (and ML), an **inner product**:

- might not be simple multiplication
- might include weights, kernels, or functions

Examples:

- Weighted inner product:

$$\langle x, w \rangle = x^\top A w$$

- Function inner product (in function space):

$$\langle f, g \rangle = \int f(t) g(t)\, dt$$

Using $\langle \cdot, \cdot \rangle$ keeps the math **abstract and reusable**.

Now apply this to your formula

You wrote:

$$x_\parallel = \mathrm{proj}_w(x) = \frac{\langle x, w \rangle}{\langle w, w \rangle}\, w$$

What does this mean geometrically?

- $\langle x, w \rangle$ measures **how much of x points in the direction of w**
- $\langle w, w \rangle = \|w\|^2$ is the squared length of w
- The ratio gives a **scalar coefficient**
- Multiplying by w gives the **projection of x onto the line spanned by w**

Tiny numerical example (to make it concrete)

Let:

$$x = (2, 1), \quad w = (1, 1)$$

Compute inner products:

$$\langle x, w \rangle = 2 \cdot 1 + 1 \cdot 1 = 3$$

$$\langle w, w \rangle = 1^2 + 1^2 = 2$$

Projection:

$$x_{\parallel} = \frac{3}{2}(1,1) = (1.5, 1.5)$$

Why this notation shows up everywhere in ML

Because:

- linear models use $w^\top x$
- attention uses $q^\top k$
- kernels use $\langle \phi(x), \phi(y) \rangle$
- gradient descent updates depend on inner products

So $\langle \cdot, \cdot \rangle$ is the **unifying language**.

Exam-safe one-liner (memorize this)

> $\langle x, w \rangle$ denotes the inner product (dot product in $\mathbb{R}^n$), measuring how much vector x aligns with vector w; it generalizes to weighted and kernelized inner products.

If you want, I can also explain:

- why projection minimizes distance
- how this connects to least squares
- why attention is an inner-product machine

Just say the word 👍

ECE590_HW2.pdf
PDF

is this homework related to transformer? I have no clue how transformer is related here

This is a **very reasonable confusion**, and you're not missing anything obvious. Let me be very clear and honest:

> **This homework is *not* directly about Transformers.**
> But it is **foundational to the exact Transformer theory you just learned in Lectures 10–11.**

I'll explain **precisely how they are connected**, and **why the homework itself does not mention Transformers at all**.

Short answer (so you can breathe)

- ❌ This HW is **not asking you to implement or analyze a Transformer**
- ✅ This HW is teaching the **mathematical substrate** that Lectures 10–11 *reinterpret* as Transformer behavior

If Lectures 10–11 are the **"aha"**, this HW is the **algebra underneath the aha**.

What this homework is *actually* about

Let's look at the header:

> *"We will consider gradient descent for three types of data models, $p_\theta(Y|X)$"*
> ECE590_HW2

So the HW is about:

- probabilistic models
- negative log-likelihood
- gradients wrt parameters
- linear / logistic / softmax models
- feature maps $\psi(x)$

This is **classical statistical learning theory**, not Transformers.

Why this feels disconnected from Transformers

Because in **Lectures 10–11**, the instructor did something subtle:

> They **replaced "gradient descent on parameters"**
> with **"gradient-descent-like updates on representations / functions"**

But that only makes sense **if you already understand**:

- how GD works for linear, logistic, softmax models
- how feature maps work
- how kernels arise from inner products
- how gradients depend on residuals

That's exactly what this HW is drilling.

The deep connection (this is the key insight)

Homework view (classical)

You are doing:

$$\theta \leftarrow \theta - \alpha \nabla_\theta \ell(\theta)$$

Where:

- $\theta = (w, b)$ or (A, b)
- learning happens by **changing parameters**

Lecture 10–11 view (Transformer reinterpretation)

They ask:

> What if the **parameters are fixed**, but the **function value updates instead**?

And they show:

$$f_{k+1}(x_j) = f_k(x_j) + \alpha \sum_i (y_i - f_k(x_i))\, \kappa(x_i, x_j)$$

Which comes **directly** from the same gradients you are deriving in HW.

Let's map HW → Lecture 10–11 explicitly

🔷 HW Problem 1: Gaussian regression with feature map

From HW:

$$f_\theta(x) = w^T \psi(x) + b$$

This is exactly:

- the **linear-in-feature-space model**
- that Lecture 10 later kernelizes
- and then reinterprets as attention

The gradients you compute:

$$\nabla_w \ell = \sum_i (f(x_i) - y_i)\psi(x_i)$$

become, in Lecture 10:

$$\Delta f(x_j) = \sum_i (y_i - f(x_i))\langle \psi(x_i), \psi(x_j)\rangle$$

That inner product is **attention**.

🔷 HW Problem 2: Logistic regression

Here you compute:

$$\nabla_w \ell = \sum_i (\sigma(z_i) - y_i)\psi(x_i)$$

Lecture connection:

- residual-driven updates
- nonlinearity inside the loss
- same structure as Transformer logits + softmax

This prepares you for **softmax attention and classification heads**.

🔷 HW Problem 3: Softmax with latent function $f(x)$

This is the **closest to the exam + Lecture 11**.

HW defines:

$$p(Y=y|x) = \frac{e^{w_y^T f(x)}}{\sum_c e^{w_c^T f(x)}} \quad \text{with} \quad f(x) = A\psi(x) + b$$

Lecture 11 then does:

- GD on $f(x)$ directly
- shows first-layer attention ≈ this update
- **without updating A**

So HW teaches:

> how gradients flow **through** $f(x)$

Lecture 11 says:

> the Transformer **implements that flow internally**

Why the HW does NOT say "Transformer"

Because the instructor is doing this pedagogically:

1. **HW:** Make sure you can compute gradients correctly
2. **Lectures 10–11:** Reinterpret those gradients as attention updates
3. **Exam:** Ask you to recognize the connection abstractly

If HW mentioned Transformers, it would:

- collapse two learning goals into one
- confuse students who haven't seen the reinterpretation yet

Exam-relevant takeaway (VERY IMPORTANT)

If asked something like:

> "Is this homework about Transformers?"

A perfect answer would be:

> No. The homework focuses on classical gradient descent for probabilistic models in feature space. However, the gradients derived here are later reinterpreted in Lectures

10–11 as functional updates implemented by attention and skip connections in Transformers.

One-sentence mental model (memorize this)

> **Homework 2 teaches how gradient descent updates parameters; Lectures 10–11 show that Transformers can implement the *same update logic* on representations instead of parameters.**

You are not missing anything.

The disconnect you feel is **intentional** and actually means you're thinking at the *right abstraction level*.

If you want next, I can:

- show **exactly** how one HW gradient becomes one attention update
- walk through **Problem 3 → Exam 1 → Transformer**
- help you decide **what to memorize vs what to understand**

Just tell me.

Sources